

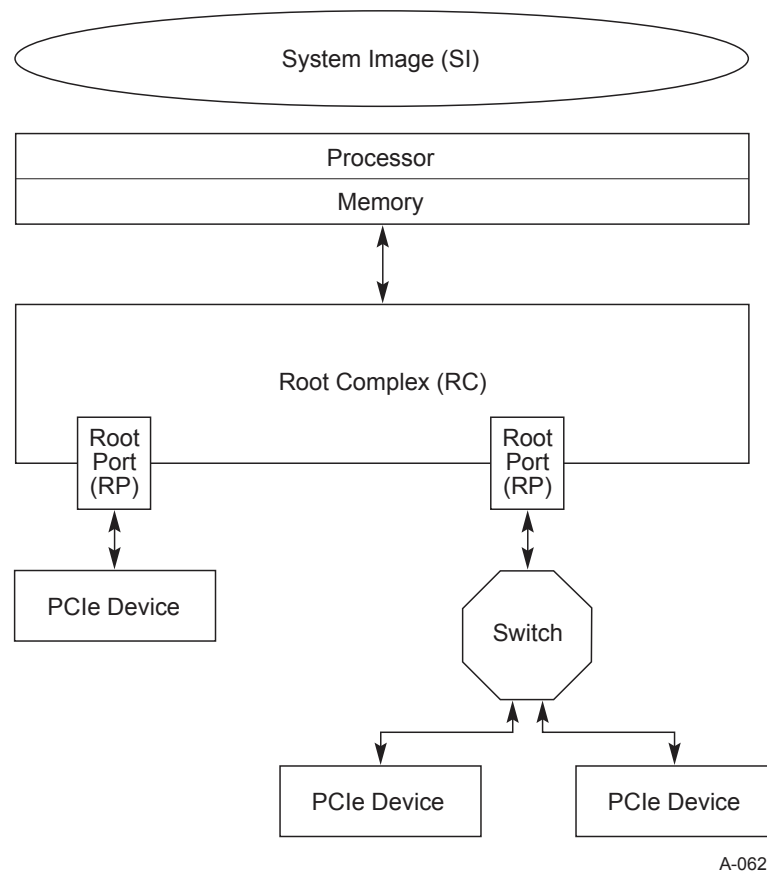
9. I/O Virtualization and Sharing §

This chapter covers two approaches to virtualization and sharing, Single Root I/O Virtualization (SR-IOV) and Scalable I/O Virtualization (SIOV). SR-IOV standardizes more of the device virtualization scenario but requires more hardware per virtualized interface than SIOV. SIOV relies on system software to fill in more of the overall virtualization scenario, thus requiring less hardware per virtualized interface, with the intention that each virtualized interface will have a reduced cost and complexity.

9.1 IOV Architectural Overview §

Within the industry, significant effort has been expended to increase the effective hardware resource utilization (i.e., application execution) through the use of virtualization technology. IOV, including Single Root I/O Virtualization and Sharing (SR-IOV) and Scalable I/O Virtualization (SIOV) enable multiple System Images (SI) to share PCI hardware resources.

To illustrate how this technology can be used to increase effective resource utilization, consider the generic platform configuration illustrated in § Figure 9-1.



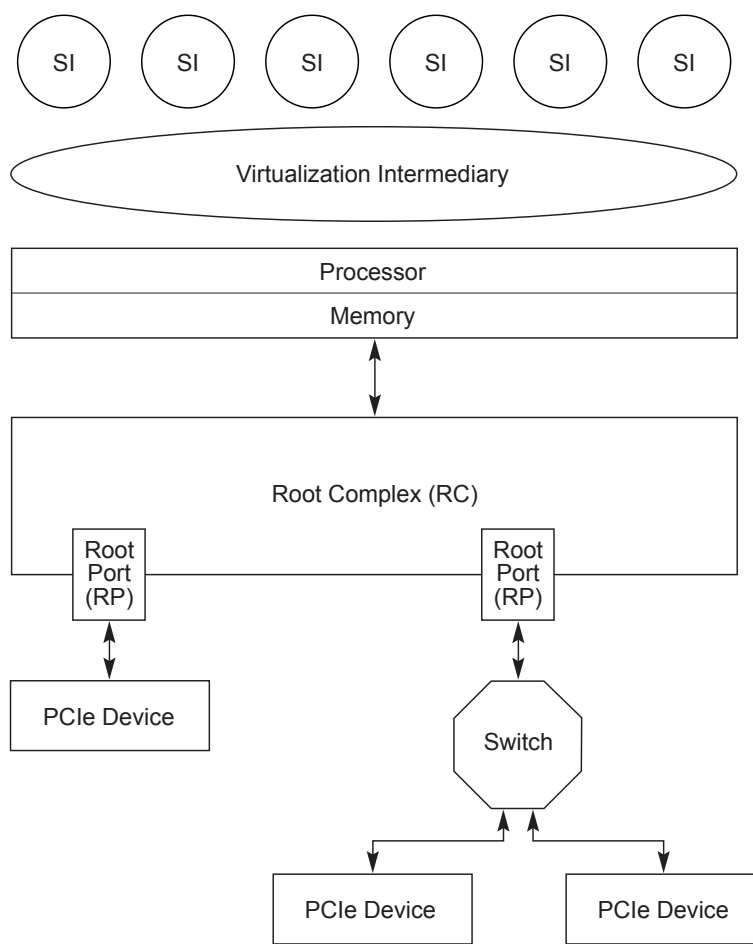
A-0622

Figure 9-1 Generic Platform Configuration §

The generic platform configuration is composed of the following components:

- PCIe Root Complex (RC), which includes:
 - Processor - general purpose, embedded, or specialized processing element
 - Memory - general purpose or embedded
 - Root Complex Integrated Endpoints (RCiEPs)
 - PCIe Root Ports (RP) - Each RP represents a separate hierarchy.
- PCIe Switch - provides I/O fan-out and connectivity
 - PCIe Device - multiple I/O device types, (e.g., network, storage, etc.)
 - System Image - software such an operating system that is used to execute applications or trusted services, (e.g., a shared or non-shared I/O device driver). In many contexts, this is referred to as a virtual machine. In other contexts, this might be referred to as a container.

In order to increase the effective hardware resource utilization without requiring hardware modifications, multiple SI can be executed. Software termed a Virtualization Intermediary (VI— often referred to as a hypervisor or virtual machine manager) is interposed between the hardware and the SI as illustrated in § Figure 9-2.



A-0623

Figure 9-2 Generic Platform Configuration with a VI and Multiple SI §

The VI takes sole ownership of the underlying hardware. Using a variety of methods outside of the scope of this specification, the VI abstracts the hardware to present each SI with its own virtual system. The actual hardware resources available to each SI can vary based on workload or customer-specific policies. While this approach works well for many environments, I/O intensive workloads can suffer significant performance degradation. Each I/O operation - inbound or outbound - must be intercepted and processed by the VI adding significant platform resource overhead.

IOV provides tools to reduce these platform resources overheads. The benefits of IOV are:

- The ability to eliminate VI involvement in main data movement actions - DMA, Memory space access, interrupt processing, etc. Elimination of VI interception and processing of each I/O operation can provide significant application and platform performance improvements.
- Standardized method to control IOV resource configuration and management through Single Root PCI Manager (SR-PCIM).
 - Due to a variety of implementation options - system firmware, VI, operating system, I/O drivers, etc. - SR-PCIM implementation is outside the scope of this specification.
- The ability to reduce the hardware requirements and associated cost with provisioning potentially a significant number of I/O Functions within a device.
- The ability to integrate IOV with other I/O virtualization technologies such as Address Translation Services (ATS), **Address Translation and Protection Table (ATPT)** technologies, and interrupt remapping technologies to create robust, complete I/O virtualization solutions. Together, the ATPT and interrupt remapping technologies are used by the Translation Agent, often referred to as an IOMMU, or I/O Memory Management Unit.

§ Figure 9-3 illustrates an example SR-IOV capable platform.

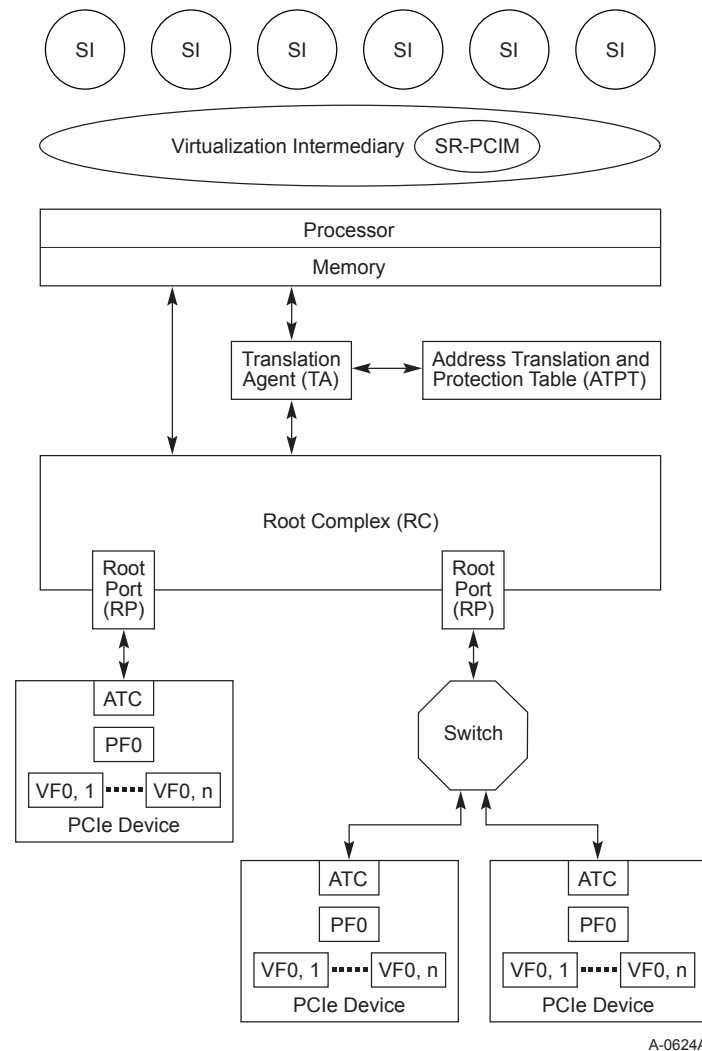


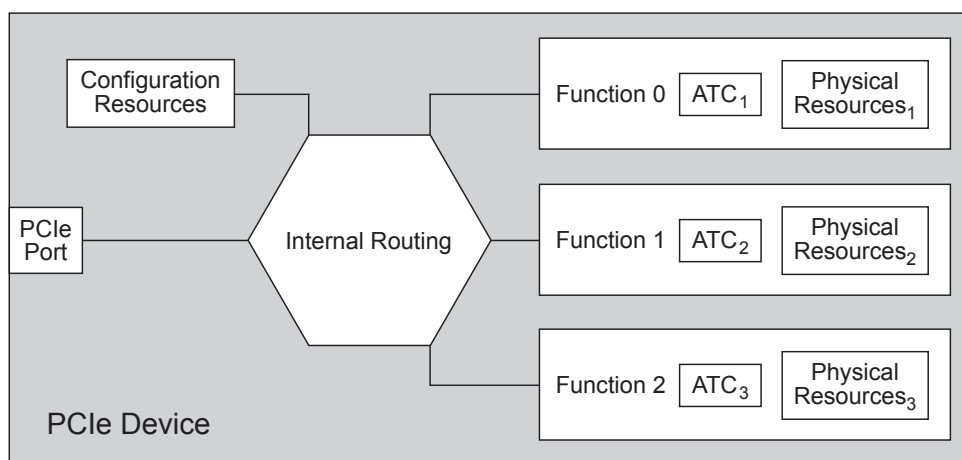
Figure 9-3 Generic Platform Configuration with SR-IOV and IOV Enablers §

The IOV generic platform configuration is composed of the following additional functional elements:

- SR-PCIM - Software responsible for:
 - the configuration of the SR-IOV Extended Capability and SIOV Extended Capability,
 - management of Physical Functions, Virtual Functions, and SDIs, and
 - processing of associated error events and overall device controls such as power management and hot-plug services.
- Optional Translation Agent (TA) - A TA is hardware or a combination of hardware and software responsible for translating an address within a PCIe transaction into the associated platform physical address. A TA may contain an Address Translation Cache (ATC) to accelerate translation table access. A TA may also support Address Translation Services (ATS) which enables a PCIe Function to obtain address translations *a priori* to DMA access to the associated memory. See § Chapter 10. for more details on ATS benefits and operation.
- Optional Address Translation and Protection Table (ATPT) - An ATPT contains the set of address translations accessed by a TA to process PCIe requests - DMA Read, DMA Write, or interrupt requests. See Address Translation Services (§ Chapter 10.) for additional details.

- In PCIe, interrupts are treated as memory write operations. Through the combination of a Requester Identifier and the address contained within a PCIe transaction, an interrupt can be routed to any target (e.g., a processor core) transparent to the associated I/O Function.
- DMA Read and Write requests are translated through a combination of the Routing ID and the address contained within a PCIe transaction.
- Optional Address Translation Cache (ATC) - An ATC can exist in two locations within a platform - within the TA which can be integrated within or sit above an RC - or within a PCIe Device. Within an RC, the ATC enables accelerated translation look ups to occur. Within a Device, the ATC is populated through ATS technology. PCIe transactions that indicate they contain translated addresses may bypass the platform's ATC in order to improve performance without compromising the benefits associated with ATPT technology. See Address Translation Services (§ Chapter 10.) for additional details.
- Optional Access Control Services (ACS) - ACS defines a set of control points within a PCI Express topology to determine whether a TLP should be routed normally, blocked, or redirected. In a system that supports IOV, ACS may be used to prevent device Functions or SDIs assigned to the VI or different SIs from communicating with one another or a peer device. Redirection may permit a Translation Agent to translate Upstream memory TLP addresses before a peer-to-peer forwarding decision is made. Selective blocking may be provided by the optional ACS P2P Egress Control. ACS is subject to interaction with ATS. See § Section 6.12 for additional details.
- Physical Function (PF) - A PF is a PCIe Function that supports the SR-IOV Extended Capability or SIOV Extended Capability and is accessible to an SR-PCIM, a VI, or an SI.
- Virtual Device Interface – A virtual device interface is directly accessible by an SI.
 - Resources associated with the data processing in the Physical Function are available to the SI. Device configuration and control surfaces are typically restricted to a trusted software component such as a VI or SR-PCIM, which interacts with the PF in a manner that is not exposed to an SI directly.
 - A virtual device interface can be serially shared by different SIs, (i.e., a device Interface can be assigned to one SI and then reset and assigned to another SI.)
 - Virtual Function (VF) – A VF is a “light-weight” PCIe Function that is a virtual device interface. VFs are discovered and configured via the SR-IOV Extended Capability structure.
 - VFs have a Configuration Space that is a subset of that of an unvirtualized Function with a Type 0 header.
 - VFs expose registers and memory to an SI via a section of a Virtual Function BAR.
 - All VFs associated with a PF must be the same device type and class as the PF, (e.g., the same network device type or the same storage device type).
 - Scalable Device Interface (SDI) – An SDI is a virtual device interface exposed and configured via the SIOV Extended Capability structure in conjunction with SR-PCIM.
 - SDIs have no Configuration Space of their own, relying on SR-PCIM for the portions of interface management that would have been expressed as Capability Structures or in the Type 0 header if the Device Interface were implemented as a VF.
 - SDIs have no BAR of their own, relying on the VI to use page tables to map a virtual view of the SDI into an SI, where the pages mapped may be in the PF's BARs, in host RAM, or they may be entirely synthesized by the VI.
 - SDIs, as they do not express their device type and class through a Configuration Space, are not limited to exposing a device interface of the same type as the PF.
 - SDIs have a RID but do not meet the definition of a Function. Requirements in this specification for Functions do not apply to SDIs unless specifically called out.

To compare and contrast a PCIe Device with a PCIe IOV capable device, examine the following set of figures. § Figure 9-4 illustrates an example PCIe-compliant Device.



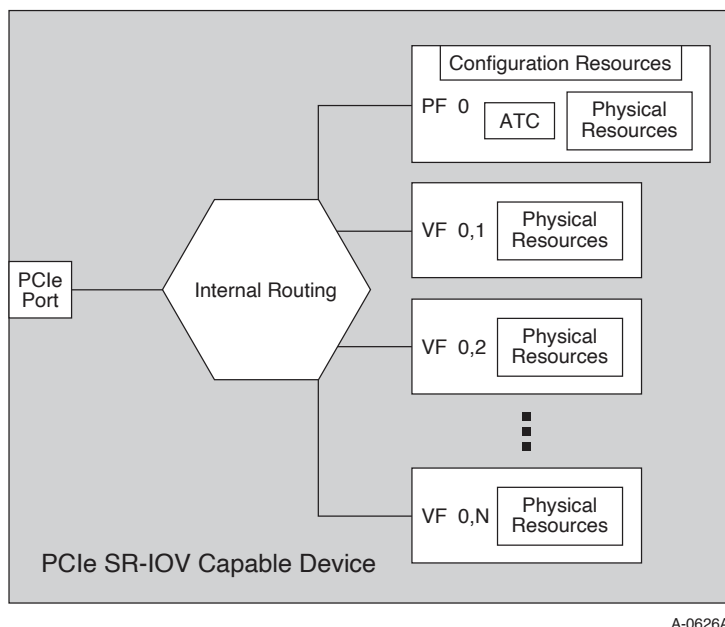
A-0625

Figure 9-4 Example Multi-Function Device §

This figure illustrates an example multi-Function PCIe Device with the following characteristics:

- The PCIe Device shares a common PCIe Link. The Link and PCIe functionality shared by all Functions is managed through Function 0.
 - While this figure illustrates only three Functions, with the use of the Alternative Routing Identifier (ARI) capability, a PCIe Device can support up to 256 Functions.
 - All Functions use a single Bus Number captured through the PCI enumeration process.
- In this example, each PCIe Function supports the ATS capability and therefore has an associated ATC to manage ATS obtained translated addresses.
- Each PCIe Function has a set of unique physical resources including a separate configuration space and BAR.
- Each PCIe Function can be assigned to an SI. To prevent one SI from impacting another, all PCIe configuration operations should be intercepted and processed by the VI.

As this figure illustrates, the hardware resources scale with the number of Functions provisioned. Depending upon the complexity and size of the device, the incremental cost per Function will vary. To reduce the incremental hardware cost, a device can be constructed using SR-IOV to support a single PF and multiple VFs as illustrated in § Figure 9-5.



A-0626A

Figure 9-5 Example SR-IOV Single PF Capable Device §

The example in § Figure 9-5 illustrates a single PF with N VFs. Key observations to note:

- The PF is a PCIe-compliant Function.
 - Initially and after a conventional reset, a PCIe Function that supports the SR-IOV capabilities defined in this specification shall have the SR-IOV capabilities disabled.
 - To discover the page sizes supported by a PF and its associated VF, the Supported Page Sizes configuration field is read. For additional information on how this field can be used to align PF or VF Memory space apertures on a system page boundary, see § Section 9.2.1.1.1.
- PF nomenclature **PF M** designates the PF at Function number M.
- VF nomenclature **VF M,N** designates the Nth VF associated with PF M. VFs are numbered starting with 1 so the first VF associated with PF M is VF M,1.
- Each VF shares a number of common configuration space fields with the PF; (i.e., where the fields are applicable to all VF and controlled through a single PF. Sharing reduces the hardware resource requirements to implement each VF).
 - A VF uses the same configuration mechanisms and header types as a PF.
 - All VFs associated with a given PF share a VF BAR set (see § Section 9.4.3.14) and share a VF Memory Space Enable (MSE) bit in the SR-IOV extended capability (see § Section 9.4.3.3.4) that controls access to the VF Memory space. That is, if the VF MSE bit is Clear, the memory mapped space allocated for all VFs is disabled.
 - The InitialVFs and TotalVFs fields (see § Section 9.4.3.5 and § Section 9.4.3.6) are used to discover the maximum number of VFs that can be associated with a PF.
 - TotalVFs and InitialVFs shall contain the same value²⁰⁵.

205. InitialVFs and TotalVFs could have had different values due to MR-IOV (now deprecated).

- Each Function, PF, and VF is assigned a unique Routing ID. The Routing ID for each PF is constructed as per § Section 2.2.4.2 . The Routing ID for each VF is determined using the Routing ID of its associated PF and fields in that PF's SR-IOV Extended Capability.
- All PCIe and SR-IOV configuration access is assumed to be through a trusted software component such as a VI or an SR-PCIM.
- Each VF contains a non-shared set of physical resources required to deliver Function-specific services, (e.g., resources such as work queues, data buffers, etc.) These resources can be directly accessed by an SI without requiring VI or SR-PCIM intervention.
- One or more VF may be assigned to each SI. Assignment policies are outside the scope of this specification.
- While this example illustrates a single ATC within the PF, the presence of any ATC is optional. In addition, this specification does not preclude an implementation from supporting an ATC per VF within the Device.
- Internal routing is implementation specific.
- While many potential usage models exist regarding PF operation, a common usage model is to use the PF to bootstrap the device or platform strictly under the control of a VI. Once the SR-IOV Extended Capability is configured enabling VF to be assigned to individual SI, the PF takes on a more supervisory role. For example, the PF can be used to manage device-specific functionality such as internal resource allocation to each VF, VF arbitration to shared resources such as the PCIe Link or the Function-specific Link (e.g., a network or storage Link), etc. These policy, management, and resource allocation operations are outside the scope of this specification.

Another example usage model is illustrated in § Figure 9-6. In this example, the device supports multiple PFs each with its own set of VFs.

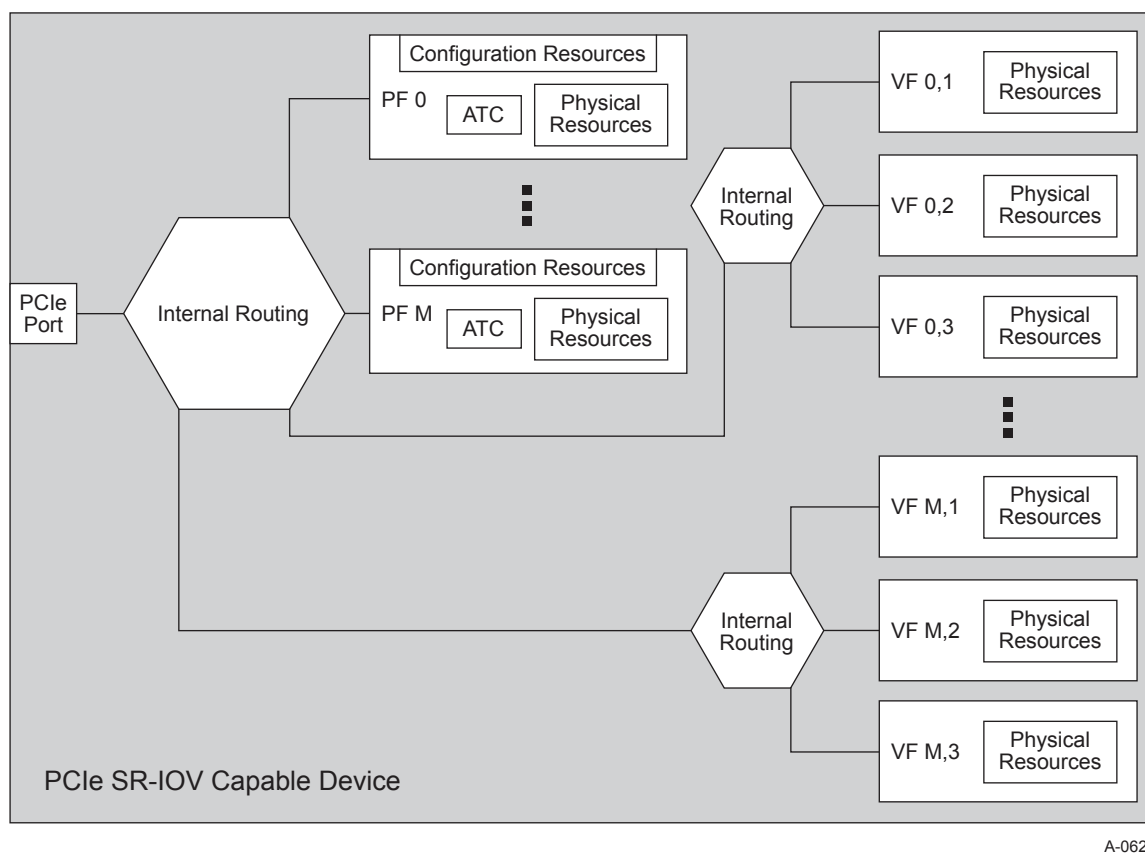


Figure 9-6 Example SR-IOV Multi-PF Capable Device §

Key observations to note:

- Each PF can be assigned one or more VFs. The number of VFs per PF is not required to be identical for all PFs within the device.
- The ARI Extended Capability enables Functions to be assigned to Function Groups and defines how Function Group arbitration can be configured. PFs and VFs can be assigned to Function Groups and take advantage of the associated arbitration capabilities. Within each Function Group, though, arbitration remains implementation specific.
- Internal routing between PFs and VFs is implementation specific.
- For some usage models, all PFs may be the same device type; (e.g., all PFs deliver the same network device or all deliver the same storage device functionality). For other usage models, each PF may represent a different device type; (e.g., in § Figure 9-6, one PF might represent a network device while another represents an encryption device).
 - In situations where there is a usage model dependency between device types, such as for each VF that is a network device type, each SI also requires a VF that is an encryption device type. The SR-IOV Extended Capability provides a method to indicate these dependencies. The policies used to construct these dependencies as well as assign dependent sets of VF to a given SI are outside the scope of this specification.

As seen in the prior example, the number of PF and VF can vary based on usage model requirements. To support a wide range of options, an SR-IOV Device can support the following number and mix of PF and VF:

- Using the Alternative Routing Identifier (ARI) capability, a device may support up to 256 PFs. Function Number assignment is implementation specific and may be sparse throughout the 256 Function Number space.
- A PF can only be associated with the Device's captured Bus Number as illustrated in § Figure 9-7.
- SR-IOV Devices may consume more than one Bus Number. A VF can be associated with any Bus Number within the Device's Bus Number range - the captured Bus Number plus any additional Bus Numbers configured by software. See § Section 9.2.1.2 for details.
 - Use of multiple Bus Numbers enables a device to support a very large number of VFs - up to the size of the Routing ID space minus the bits used to identify intervening buses.
 - If software does not configure sufficient additional Bus Numbers, then the VFs implemented for the additional Bus Numbers may not be visible.

IMPLEMENTATION NOTE: FUNCTION CO-LOCATION §

The ARI Extended Capability enables a Device to support up to 256 Functions - Functions, PFs, or VFs in any combination - associated with the captured Bus Number. If a usage model does not require more than 256 Functions, implementations are strongly encouraged to co-locate all Functions, PFs, and VFs within the captured Bus Number and not require additional Bus Numbers to access VFs.

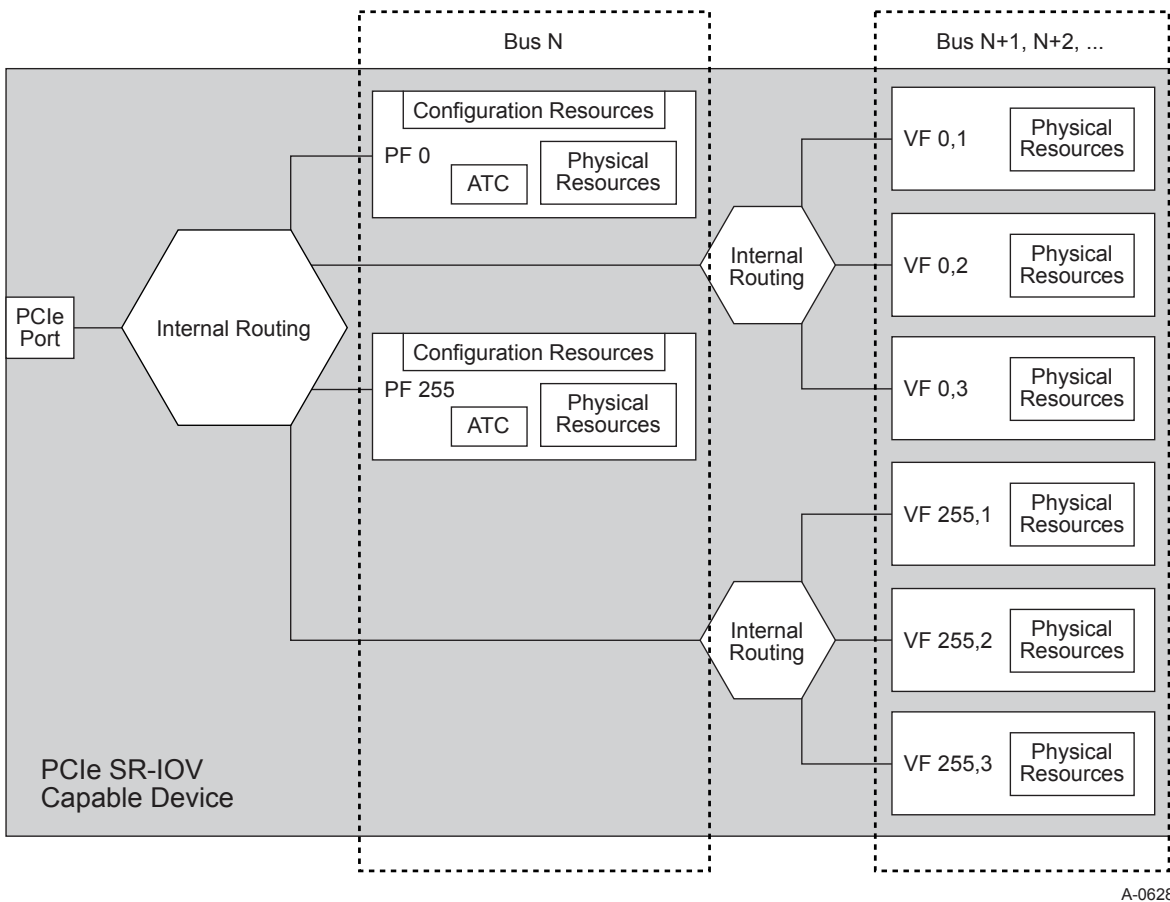


Figure 9-7 Example SR-IOV Device with Multiple Bus Numbers §

In this example, § Figure 9-8, a device implementation may mix any number of Functions, PFs, and VFs.

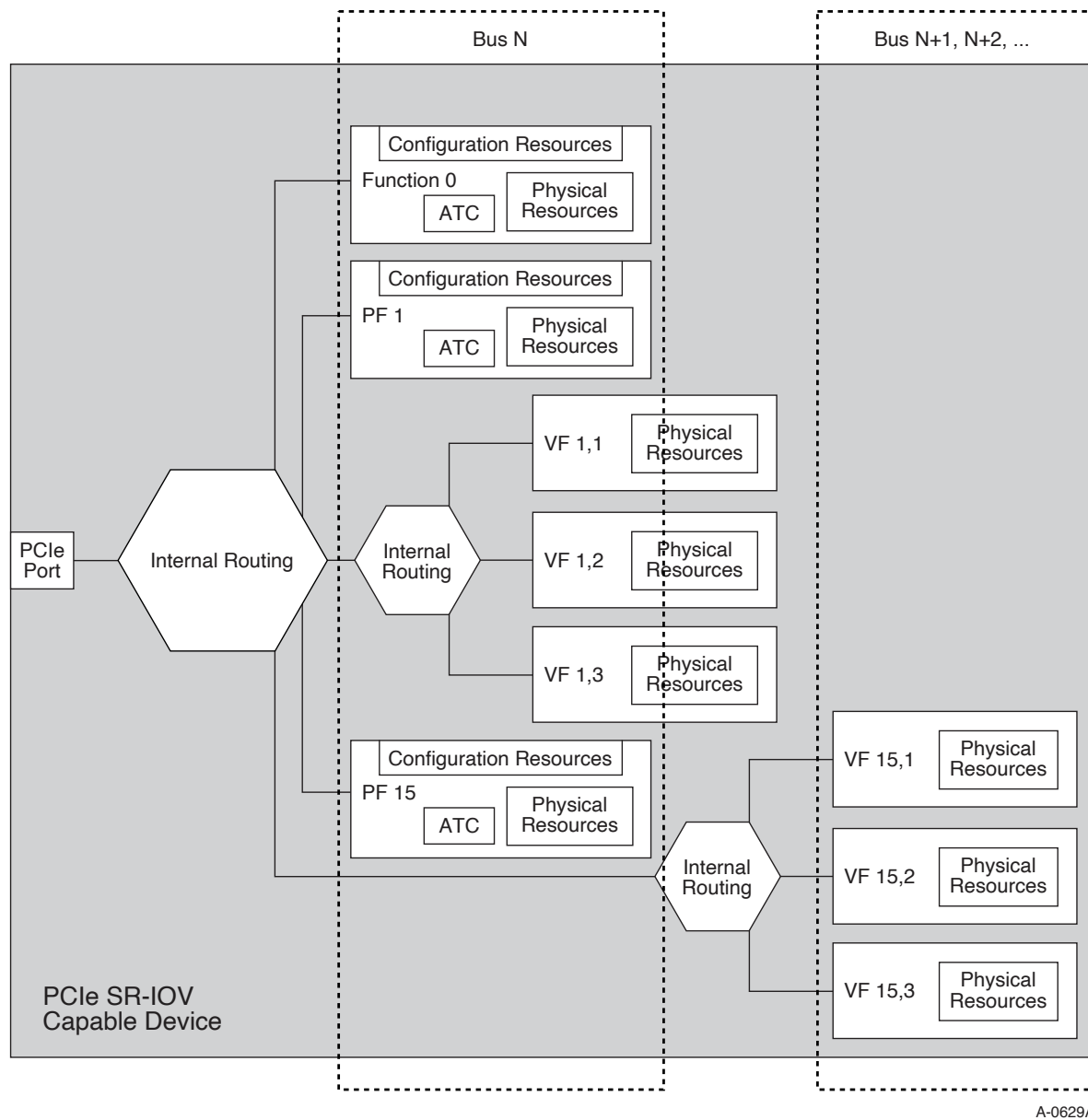


Figure 9-8 Example SR-IOV Device with a Mixture of Function Types §

Key observations to note:

- Each Device must contain a Function 0. Function 0 may be a PF (i.e., it may include the SR-IOV extended capability).
- Any mix of Functions can be associated with the captured Bus Number.
 - Non-VFs can only be associated with the captured Bus Number.
- If the ARI Extended Capability is supported, Functions can be assigned to Function Groups. The assignment policy is outside the scope of this specification. If the ARI Extended Capability is not supported, Functions can still use the Function arbitration capabilities as defined in § Section 6.3.3.4 .

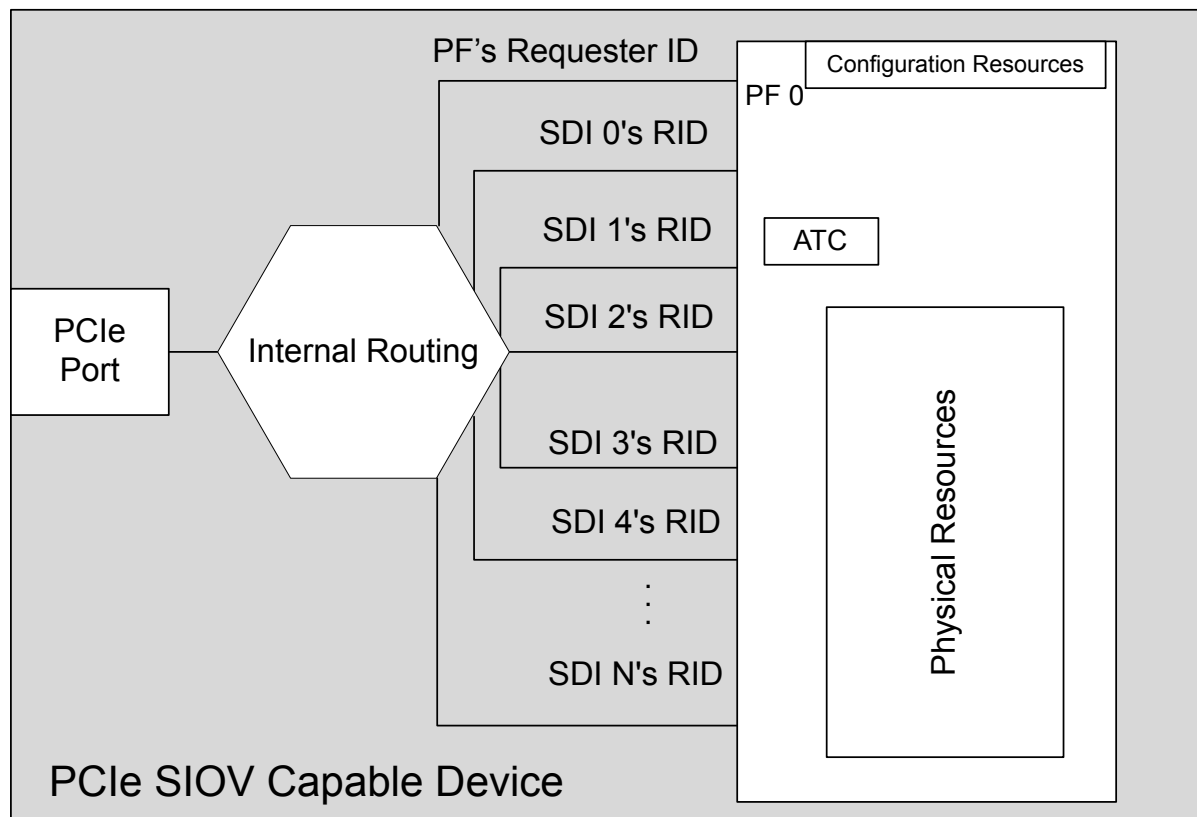


Figure 9-9 Example SIOV Capable Device §

The example in § Figure 9-9 shows a device implementing Scalable IOV, contrasting § Figure 9-5. It illustrates a similar device but using Scalable Device Interfaces instead of Virtual Functions. For the purposes of the example, assume that the SDIs are in use by SIs running on top of a VI.

Key observations to note:

- The PF is a Function.
 - Initially and after a conventional reset, a PCIe Function that supports Scalable IOV shall have the SIOV capabilities disabled.
 - Discovery of page sizes supported by the PF is analogous to SR-IOV.
- All SDI Configuration Space access is assumed to be fully virtualized by a VI.
- Each SDI is a composition of physical resources within the PF.
 - Virtual BARs, as viewed from an SI, are mappings onto sets of pages within the PF BAR, if the SI requires the SDI to be presented by the VI as a Function. Ranges within the virtual BAR may also be provided entirely by the VI, using SR-PCIM to handle reads and writes.
 - Virtual MSI or MSI-X is a mapping onto the internal interrupt management logic, if the SDI generates interrupts. Presentation of an MSI Capability or an MSI-X Capability to an SI may leverage SR-PCIM, instead of being implemented in device hardware or firmware.
 - Device-specific resources can be either dedicated to an SDI or shared across SDIs.
- Traffic on behalf of a specific SDI is tagged with a RID dedicated to that SDI.

IMPLEMENTATION NOTE:

SDI MSI STATE MANAGEMENT §

Scalable Device Interfaces may need to generate interrupts, and when they do, the SDIs have the same underlying requirements that any Function would. The device should be able to store the state implied by the interrupt model underlying MSI or the interrupt model underlying MSI-X. The device should store this state somewhere, but this specification does not stipulate where. SR-PCIM handles mapping the SI's interrupt requirements onto device-internal state.

One potential strategy for this is to map MSI-X interrupt configurations from the SI onto MSI-X table entries in the PF's BAR, where an Interrupt Message will be sent using the contents of a specific entry, using the SDI's RID. Other strategies might involve storing state equivalent to MSI or MSI-X in different parts of the device.

IMPLEMENTATION NOTE:

SHARING PAGES IN THE PF BAR WITH MULTIPLE SDIS §

SR-PCIM may choose to share pages of PF BAR space with multiple SDIs, but this comes with restrictions. If a page is used with multiple SDIs, the device will have no indication of which SDI is being read or written, since reads and writes may be address-routed and may not carry the RID associated with the SDI. Any side effects of a read or a write that would cause traffic to be initiated by the device will not be able to determine the RID of the associated SDI.

Devices implementing TDISP face additional complexity, as some accesses may originate from within a Trusted Execution Environment (TEE), while others originate from outside it. Moreover, TSMs and VIs may not support mapping common pages into TVMs.

9.1.1 RID in Endpoints where a PF contains an SIOV Extended Capability Structure §

To allow SDIs to be used within the context of separate address spaces, an Endpoint employing SIOV uses a unique RID for each SDI. This RID must contain a Bus Number that has been allocated to the Endpoint device containing the PF with an SIOV Extended Capability structure. An SDI's RID must not be used by any Function or any other SDI.

SR-IOV assigns a Virtual Function a RID based on a formula where an offset and a stride is used to compute the RID of the VF. This is an option with SIOV, though it may not allow the device to efficiently use all of the RID space, perhaps because the upstream bridges are Flattening Portal Bridges. Similarly, some PCIe Switches can be attached to multiple Root Complexes, simultaneously (though this is a vendor-specific extension, and when this technique is employed, traffic is often routed to one Root Complex or another based on partitioning the RID space, perhaps by Bus Number). In these cases, maximizing the number of available SDIs involves using all available RIDs, which requires a dynamic association between RIDs and SDIs.

This choice is expressed through the SIOV Extended Capability structure. The first option allows the device to share or reuse RID mapping logic from existing SR-IOV devices. Just as with SR-IOV, the SDIs RID is calculated by multiplying the SDI number by a supplied stride and then applying an offset to the PF's RID.

The other option is to allow the VI to supply the RID for the SDI. If this is employed, after Bus Numbers have been assigned to all the relevant bridges during the boot process and before any SDIs are placed into use, the set of potential RIDs can be calculated with the following algorithm:

1. Identify the set of Bus Numbers passed through the upstream bridge by starting with its Secondary and Subordinate Bus Numbers and adding in all buses implied by the configuration of a Flattening Portal Bridge capability, if present.
2. Calculate the set of RIDs available to the Endpoint on each of these logical buses. Each bus has 256 RIDs available.
3. Remove from this set of RIDs any Function enumerable by reading Configuration Space of Function 0 on the Secondary Bus and continuing in order as implied by the Multi-Function Device bit in its Header Type register.
4. Further remove from this set of RIDs any Function enumerable by the Alternative Routing Interpretation (ARI).
5. Find each of the PFs within this Endpoint, other than the one for which this algorithm is targeting.
 - a. If that PF has an SR-IOV Extended Capability structure and no Scalable IOV Extended Capability structure, calculate the set of RIDs used by its VFs and remove them from the target set.
 - b. If that PF has a Scalable IOV Extended Capability structure and it uses the Strided RID mapping, calculate the set of RIDs it would use for SDIs and remove them from the target set.
 - c. If that PF has an SIOV Extended Capability and it does not use the Strided RID mapping, remove any RIDs currently in use by it.
6. Find all Phantom Functions and Shadow Functions and remove them from the set of RIDs.

The resulting set of RIDs contains valid values for use by SDIs.

9.1.2 SDI Reset §

The SDI reset mechanism enables software to quiesce and reset the SDI functionality. Three example usage models illustrate the need for this feature:

- In some systems, it is possible that the software entity that controls the SDI will cease to operate normally. To prevent data corruption, it is necessary to stop all PCI Express and external I/O (not PCI Express) operations being performed by the SDI.
- In a partitioned environment where the SDI is migrated from one partition to another, it is necessary to ensure that no residual “knowledge” of the prior partition be retained by the SDI, for example, a user’s secret information entrusted to the first partition but not to the second.
- When system software is taking down the software stack for the SDI and then rebuilding that stack, it is sometimes necessary to return the state to an uninitialized state before rebuilding the SDI’s software stack.

Each SDI must be independently resettable without affecting the operation of other SDIs. Reset of an SDI is performed through device-specific interfaces. The SDI reset causes the device to abort (discard) all in flight and accepted operations to the SDI by specific domain and reset SDI specific configuration on the device to a known state:

- Unarchitected SDI state configured through the PF or SR-PCIM interface is permitted to persist across SDI instantiations.
- Architected SDI state configured through the PF or SR-PCIM interface is permitted to be restored to earlier values or set to new values. That is, when the SDI is enabled again, the new instance of SDI is permitted to have a different set of architected capabilities and/or their attributes (e.g., RID, Memory Space resources, interrupt message resources) from the previous instance.

The SDI reset is conceptually equivalent to an FLR. In response to the SDI reset,

- The SDI must not give the appearance of an initialized adapter with an active host on any external interfaces controlled by that SDI. The steps needed to terminate activity on external interfaces are outside of the scope of this specification.
 - For example, a network adapter must not respond to queries that would require adapter initialization by the host system or interaction with an active host system, but is permitted to perform actions that it is designed to perform without requiring host initialization or interaction.
- The SDI must not retain within itself software readable state that potentially includes secret information associated with any preceding use of the SDI. Main host memory assigned to the SDI must not be modified by the SDI.
 - For example, an SDI with internal memory readable directly or indirectly by host software must clear or randomize that memory.

An SDI reset must ensure that the reset is not reported as complete until all of the following conditions are satisfied:

- All DMA write operations by the SDI are drained or aborted.
- All DMA read operations by the SDI have completed or aborted.
- All interrupts from the SDI have been generated.
- If the SDI is capable of Address Translation Service (ATS), all ATS requests by the SDI have completed or aborted.
- If the SDI is capable of Page Request Service (PRS), no more page requests will be generated by the SDI. Additionally, either page responses have been received for all page requests generated by the SDI or the SDI will discard page responses for any outstanding page requests by the SDI.

While the SDI reset is in progress:

- If a Request arrives, the Request is permitted to be silently discarded (following update of flow control credits) without logging or signaling it as an error.
- If a Completion arrives, the Completion is permitted to be handled as an Unexpected Completion or to be silently discarded (following update of flow control credits) without logging or signaling it as an error.

9.2 SR-IOV Initialization and Resource Allocation §

9.2.1 SR-IOV Resource Discovery §

The following sections describe how software determines that a Device is SR-IOV capable and subsequently identifies VF resources through Virtual Function Configuration Space.

9.2.1.1 Configuring SR-IOV Capabilities §

This section describes the fields that must be configured before enabling a PF's IOV Capabilities. The VFs are enabled by Setting the PF's VF Enable bit (see § Section 9.4.3.3.1) in the SR-IOV extended capability.

The NumVFs field (see § Section 9.4.3.7) defines the number of VFs that are enabled when VF Enable is Set in the associated PF.

9.2.1.1.1 Configuring the VF BAR Mechanisms §

This section describes how the VF BARs are configured to map memory space. VFs do not support I/O Space and thus VF BARs shall not indicate I/O Space.

The System Page Size field (see § Section 9.4.3.13) defines the page size the system will use to map the VF's PCIe memory addresses when the PF's IOV Capabilities are enabled. The System Page Size field is used by the PF to align the Memory space aperture defined by each VF BAR to a system page boundary. The value chosen for the System Page Size must be one of the Supported Page Sizes (see § Section 9.4.3.12) in the SR-IOV extended capability.

The behavior of VF BARs is the same as the normal PCI Memory Space BARs (see § Section 7.5.1.2.1), except that a VF BAR describes the aperture for each VF, whereas a PCI BAR describes the aperture for a single Function. The attributes for some of the bits in the VF BARs are affected by the VF Resizable BAR Extended Capability (see § Section 7.8.7) if it is implemented.

- The behavior described in § Section 7.5.1.2.1 for determining the memory aperture of a Function's BAR applies to each VF BAR. That is, the size of the memory aperture required for each VF BAR can be determined by writing all "1"s and then reading the VF BAR. The results read back must be interpreted as described in § Section 7.5.1.2.1 .
- The behavior for assigning the starting memory space address of each BAR associated with the first VF is also as described in § Section 7.5.1.2.1 . That is, the address written into each VF BAR is used by the Device for the starting address of the first VF.
- The difference between VF BARs and BARs described in § Section 7.5.1.2.1 is that for each VF BAR, the memory space associated with the second and higher VFs is derived from the starting address of the first VF and the memory space aperture. For any given VF_v, the starting address of its Memory space aperture for any implemented BAR_b is calculated according to the following formula:

$$\text{BAR}_b \text{ VF}_v \text{ starting address} = \text{VF BAR}_b + (v - 1) \times (\text{VF BAR}_b \text{ aperture size})$$

where VF BAR_b aperture size is the size of VF BAR_b as determined by the usual BAR probing algorithm as described in § Section 9.4.3.14 .

VF memory space is not enabled until both VF Enable and VF MSE have been Set (see § Section 9.4.3.3.1 and § Section 9.4.3.3.4). Note that changing System Page Size (see § Section 9.4.3.13) may affect the VF BAR aperture size.

§ Figure 9-10 shows an example of the PF and VF Memory space apertures.

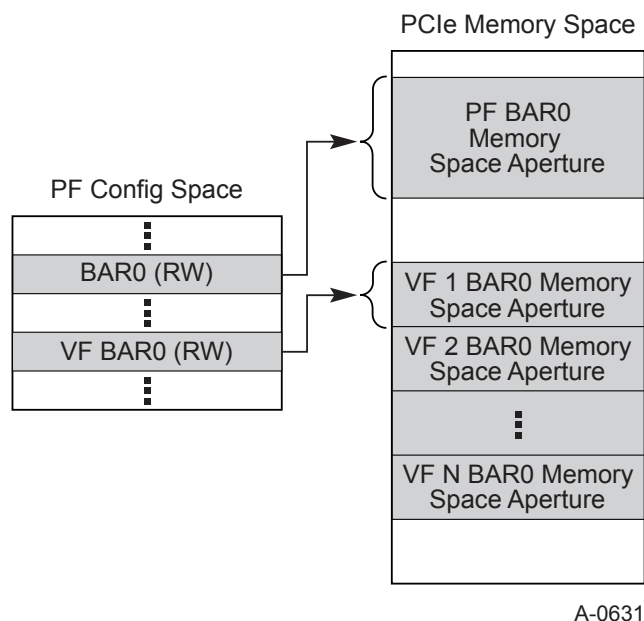


Figure 9-10 BAR Space Example for Single BAR Device §

9.2.1.2 VF Discovery §

The First VF Offset and VF Stride fields in the SR-IOV extended capability are 16-bit Routing ID offsets. These offsets are used to compute the Routing IDs for the VFs with the following restrictions:

- The value in NumVFs in a PF (§ Section 9.4.3.7) may affect the values in First VF Offset (§ Section 9.4.3.9) and VF Stride (§ Section 9.4.3.10) of that PF.
- The value in ARI Capable Hierarchy (§ Section 9.4.3.3.5) in the lowest-numbered PF of the Device (for example PF₀) may affect the values in First VF Offset and VF Stride in all PFs of the Device.
- NumVFs of a PF may only be changed when VF Enable (§ Section 9.4.3.3.1) of that PF is Clear.
- ARI Capable Hierarchy (§ Section 9.4.3.3.5) may only be changed when VF Enable is Clear in all PFs of a Device.

IMPLEMENTATION NOTE: NUMVFS AND ARI CAPABLE HIERARCHY §

After configuring NumVFs and ARI Capable Hierarchy where applicable, software may read First VF Offset and VF Stride to determine how many Bus Numbers would be consumed by the PF's VFs. The additional Bus Numbers, if any, are not actually used until VF Enable is Set.

§ Table 9-1 describes the algorithm used to determine the Routing ID associated with each VF.

Table 9-1 VF Routing ID Algorithm §

VF Number	VF Routing ID
VF 1	$(\text{PF Routing ID} + \text{First VF Offset}) \text{ Modulo } 2^{16}$
VF 2	$(\text{PF Routing ID} + \text{First VF Offset} + \text{VF Stride}) \text{ Modulo } 2^{16}$
VF 3	$(\text{PF Routing ID} + \text{First VF Offset} + 2 * \text{VF Stride}) \text{ Modulo } 2^{16}$
...	...
VF N	$(\text{PF Routing ID} + \text{First VF Offset} + (N-1) * \text{VF Stride}) \text{ Modulo } 2^{16}$
...	...
VF NumVFs (last one)	$(\text{PF Routing ID} + \text{First VF Offset} + (\text{NumVFs}-1) * \text{VF Stride}) \text{ Modulo } 2^{16}$

All arithmetic used in this Routing ID computation is 16-bit unsigned dropping all carries.

All VFs and PFs must have distinct Routing IDs. The Routing ID of any PF or VF must not overlap with the Routing ID of any other PF or VF given any valid setting of NumVFs across all PFs of a device.

VF Stride and First VF Offset are constants. Except as stated earlier in this section, their values may not be affected by settings in this or other Functions of the Device.

VFs may reside on different Bus Number(s) than the associated PF. This can occur if, for example, First VF Offset has the value 0100h. A VF shall not be located on a Bus Number that is numerically smaller than its associated PF. A VF that is located on the same Bus Number as its associated PF shall not be located on a Device Number that is numerically smaller than the PF²⁰⁶.

VFs of an SR-IOV RCiEP Device are associated with the same Root Complex Event Collector (if any) as their PF. Such VFs are not reported in the Root Complex Event Collector Endpoint Association Extended Capability of the Root Complex Event Collector.

As per § Section 2.2.6.2, SR-IOV capable Devices that are associated with an Upstream Port capture the Bus Number from any Type 0 Configuration Write Request. SR-IOV capable Devices do not capture the Bus Number from any Type 1 Configuration Write Requests. SR-IOV capable RCiEPs use an implementation specific mechanism to assign their Bus Numbers.

Note: Bus Numbers are a constrained resource. Devices are strongly encouraged to avoid leaving “holes” in their Bus Number usage to avoid wasting Bus Numbers.

All PFs must be located on the Device’s captured Bus Number.

206. SR-IOV Devices immediately below a Downstream Port always have a Device Number of 0 and thus always satisfy this condition.

IMPLEMENTATION NOTE:

VFS SPANNING MULTIPLE BUS NUMBERS §

As an example, consider an SR-IOV Device that supports a single PF. Initially, only PF 0 is visible. Software Sets ARI Capable Hierarchy. From the SR-IOV Extended Capability it determines: InitialVFs is 600, First VF Offset is 1 and VF Stride is 1.

- If software sets NumVFs in the range [0 ... 255], then the Device uses a single Bus Number.
- If software sets NumVFs in the range [256 ... 511], then the Device uses two Bus Numbers.
- If software sets NumVFs in the range [512 ... 600], then the Device uses three Bus Numbers.

PF 0 and VF 0,1 through VF 0,255 are always on the first (captured) Bus Number. VF 0,256 through VF 0,511 are always on the second Bus Number (captured Bus Number plus 1). VF 0,512 through VF 0,600 are always on the third Bus Number (captured Bus Number plus 2).

Software should configure Switch Secondary and Subordinate Bus Number fields to route enough Bus Numbers to the Device. If sufficient Bus Numbers are not available, software should reduce a Device's Bus Number requirements by not enabling SR-IOV and/or reducing NumVFs for some or all PFs of the Device prior to enabling SR-IOV.

After VF Enable is Set in some PF n , the Device must Enable VF $n,1$ through VF n,m (inclusive) where m is the smaller of InitialVFs and NumVFs. A Device receiving a Type 0 Configuration Request targeting an Enabled VF located on the captured Bus Number must process the Request normally. A Device receiving a Type 1 Configuration Request targeting an Enabled VF not located on the captured Bus Number must process the Request normally. A Device receiving a Type 1 Configuration Request targeting the Device's captured Bus Number must follow the rules for handling Unsupported Requests. Additionally, if VF MSE is Set, each Enabled VF must respond to PCIe Memory transactions addressing the memory space associated with that VF.

Functions that are not enabled (i.e., Functions for VFs above m) do not exist in the PCI Express fabric. As per § Section 2.3.1, addressing Functions that do not exist will result in Unsupported Request (UR) being returned. This includes Functions on additional Bus Numbers.

IMPLEMENTATION NOTE:

MULTI-FUNCTION DEVICES WITH PFS AND SWITCH FUNCTIONS §

SR-IOV Devices may consume multiple bus numbers. Additional bus numbers beyond the first one are consecutive and immediately follow the first bus number assigned to the device. If an SR-IOV Device also contains PCI-PCI Bridges (with Type 1 Configuration Space Headers), the SR-IOV usage must be accounted for when programming the Secondary Bus Number for those Bridges. Software should determine the last Bus Number used by VFs first and then configure any co-located Bridges to use Bus Numbers above that value.

9.2.1.3 Function Dependency Lists §

PCI Devices may have vendor-specific dependencies between Functions. For example, Functions 0 and 1 might provide different mechanisms for controlling the same underlying hardware. In such situations, the Device programming model might require that these dependent Functions be assigned to SIs as a set.

Function Dependency Lists are used to describe these dependencies (or to indicate that there are no Function dependencies). When using SR-IOV, software should assign PFs and VFs to SIs such that the dependencies are satisfied. Function Dependency Lists cannot be used with PFs when using SIOV.

See § [Section 9.4.3.8](#) for details.

9.2.1.4 Interrupt Resource Allocation §

PFs and VFs support either MSI, MSI-X interrupts, or both if interrupt resources are allocated. VFs shall not implement INTx. MSI and MSI-X interrupts are described in § [Section 6.1.4](#).

For MSI-X interrupts, special address range isolation requirements apply for the MSI-X structures in PF and VF MMIO regions. See in § [Section 7.7.2](#).

9.2.2 SR-IOV Reset Mechanisms §

This section describes how reset mechanisms defined in § [Section 6.6](#) affect Devices that support SR-IOV. It also describes the mechanisms used to reset a single VF and a single PF with its associated VFs.

9.2.2.1 SR-IOV Conventional Reset §

A Conventional Reset to a Device that supports SR-IOV shall cause all Functions (including both PFs and VFs) to be reset to their original, power-on state as per the rules in § [Section 6.6.1](#). § [Section 9.4](#) describes the behavior for the fields defined.

Note: Conventional Reset clears [VF Enable](#) in the PF. Thus, VFs no longer exist after a Conventional Reset.

9.2.2.2 FLR That Targets a VF §

VFs must support [Function Level Reset \(FLR\)](#).

Note: Software may use FLR to reset a VF. FLR to a VF affects a VF's state but does not affect its existence in PCI Configuration Space or PCI Bus address space. The VFs BAR_n values (see § [Section 9.4.3.14](#)) and VF MSE (see § [Section 9.4.3.3.4](#)) in the PF's SR-IOV extended capability, and the VF Resizable BAR capability values (see § [Section 7.8.7](#)) are unaffected by FLRs issued to the VF.

9.2.2.3 FLR That Targets a PF §

PFs must support FLR.

FLR to a PF resets the PF state as well as the SR-IOV extended capability including [VF Enable](#) which means that VFs no longer exist.

9.2.3 IOV Re-initialization and Reallocation §

If [VF Enable](#) is Cleared after having been Set, all of the VFs associated with the PF no longer exist and must no longer issue PCIe transactions or respond to Configuration Space or Memory Space accesses. VFs must not retain any architected

state after VF Enable has been Cleared (including sticky bits). For security, unarchitected VF state configured through the VF must be cleared or randomized, with the exception of persistent storage data.

Unarchitected VF state configured through the PF or SR-PCIM interface is permitted to persist across VF instances.

Architected VF state configured through the PF or SR-PCIM interface is permitted to be restored to earlier values or set to new values. That is, when VF Enabled is Set again, instances of VFs are permitted to have a different set of architected capabilities and/or their attributes (e.g., MSI-X Table size) from previous instances.

9.3 SIOV Initialization and Resource Allocation §

9.3.1 SIOV Resource Discovery §

The following sections describe how software determines that a Device is SIOV capable and outlines the process for enabling, configuring and managing SDIs.

9.3.1.1 Configuring SIOV Capabilities §

In contrast to SR-IOV, SIOV hardware does not expose Configuration Space for each SDI. As such, there is no analogous configuration of the device's capabilities. Each individual SDI may be separately configured, enabled and disabled by the VI/SR-PCIM via an implementation-specific mechanism. If system software wishes to disable the PF and all the SDIs, it can use a conventional reset or a Function-Level Reset for this.

The number of available SDIs, and thus the number of Bus Numbers required for the Endpoint device, can be derived from the SIOV Extended Capability structure.

9.3.1.1.1 Configuring an SDI §

SIOV minimizes the complexity of a device by allowing device designers to implement internal partitioning and virtualization in ways that rely on internal knowledge of the device's design. SR-PCIM is expected to include device-specific extensions to accommodate discovery and configuration. Since SRPCIM itself isn't architected within this specification, and since the internal structure of various VIs differs quite notably, these extensions are not defined in this document.

9.3.1.1.1.1 Behavior of a VI §

To support SIOV, a VI provides several sorts of mechanisms. The exact composition of these differs between VI implementations. They are listed here not to specify them, but to allow device designers to understand which mechanisms can be depended upon. Without any notion of what a VI might provide, however, device designers would be forced to implement many things in hardware, perhaps increasing the cost and complexity.

- A VI provides a mechanism for a device-specific extension to SR-PCIM.

Logically, this mechanism is an extension of the device driver for the device, though it may be packaged separately from the device driver and it may run in a different process context or even in a different SI.

- The device-specific extension to SR-PCIM provides read and write mechanisms for Configuration Space.

When the SDI is to be presented as a Function to an SI, these mechanisms, along with other parts of SR-PCIM, allow the SI to interact with the SDI as if it were a Function with a Type 0 Header. It is permitted for VI to present an SDI with a different Type 0 Header (e.g., Vendor ID, Device ID, Class Code, Subsystem Vendor ID) and PCI Express Capabilities than its associated PF.

- The device-specific extension to SR-PCIM provides a list of pages of memory (in the sizes implied by the System Page Size register) that make up each virtual BAR, and how these map onto the hardware or onto the extension itself.

When the SDI is presented as a Function, an SI interacts with it as if it had the BARs of a Function with a Type 0 Header. To do this, each page of memory is mapped onto a page of the PF BAR or the page is marked invalid in the page tables. When pages are invalid, the processor traps on access to them. This trapped access can be redirected to software, that supplies the values which would be read or written to the virtual BAR. In general, this process is known as either “emulation” or “full virtualization,” though the second term sometimes only applies to emulation of an entire device. By emulating some pages of a virtual BAR in software, a device designer can avoid replicating the setup and teardown parts of the hardware control surface into all of the SDIs.

- If MSI-X is employed, the device-specific extension provides for emulating the MSI-X table.

9.3.1.1.1.2 Requirements of an SI §

Any PCI Express Endpoint that supports TDISP may depend on an SI for specific functionality. The device drivers within the SIs optionally move the TDIs that the Endpoint supports from the CONFIG_UNLOCKED state into the CONFIG_LOCKED state, followed by a query for a DEVICE_INTERFACE_REPORT. If the fields in the DEVICE_INTERFACE_REPORT contain acceptable values for a secure configuration, the drivers move the TDI into the RUN state, which allows the SI to bring the TDI into the SI's Trusted Computing Base.

For an Endpoint that supports both TDISP and Scalable IOV, an SI accounts for the parts of the device interface that are implemented in SR-PCIM, including Configuration Space and any part of the initialization that isn't implemented in the hardware interface of the SDI. Since SR-PCIM may sit outside of the Trusted Computing Base, the device driver initializes the device interface before moving from CONFIG_UNLOCKED to CONFIG_LOCKED. Then, once the TDI is locked, the device interface report can be evaluated and then the TDI is moved to the RUN state and interactions with the SDI can be trusted.

- An SI allows a device driver for an SDI within the SI to optionally interact with that SDI before moving the TDI from CONFIG_UNLOCKED to CONFIG_LOCKED.
- An SI allows a device driver for an SDI to evaluate the DEVICE_INTERFACE_REPORT before moving a TDI from CONFIG_LOCKED to RUN.

9.3.1.2 MSI/MSI-X Interrupt Message Resource Allocation §

An MSI/MSI-X interrupt message is a Memory Write Request with system-specific values that differentiate it from other DMA Memory Write Requests. An interrupt message resource is the collection of system-specific values, such as address and data, needed to create the Memory Write Request. This specification puts no meaning on the specific values or how they are assigned/derived for interrupt message resources. Like any other Memory Write Request from an SDI, this memory write must use the RID of the SDI.

Unlike BAR space resources, interrupt message resources are not required to be assigned initially to the PF via an architected mechanism (MSI or MSI-X) and subsequently delegated to an SDI, though an implementation is permitted to choose such an approach. In fact, SDIs have no requirement to store interrupt message resource values tuples (address/data/etc.) in a processor-visible address space like Configuration Space (as MSI does) or Memory Space (like MSI-X does).

If they are stored in Memory Space, there is no requirement that they be stored contiguously or in special address ranges that can be isolated (like MSI-X requires).

The VI is permitted to present an SDI to an SI as a Function with a Type 0 Header, virtualizing the parts of the device interface that are not implemented in device hardware or firmware. If the VI presents an SDI as a Function with a Type 0 Header, it must present MSI Capability or MSI-X Capability structures and intercept any access to interrupt configuration within the SI, redirecting it to SR-PCIM. If the VI does not present an SDI as a Function with a Type 0 Header, SR-PCIM must provide alternative software interfaces supplying this functionality. SR-PCIM is free to use any mechanism the device exposes.

- A PF with an SIOV Extended Capability is permitted to expose functionality isomorphic with MSI on each SDI, in which case it must store an address value for each SDI, a base data value for each SDI, and an index for each interrupt in use by the SDI, to be added to the base data value when generating an interrupt message.
- A PF with an SIOV Extended Capability is permitted to expose functionality isomorphic with MSI-X on each SDI, in which case it must store interrupt message resource values (address/data/etc.) for each interrupt in use by the SDI.

9.4 SR-IOV Configuration §

9.4.1 SR-IOV Configuration Overview §

This section provides SR-IOV-added requirements for implementing PFs and VFs.

PFs are discoverable in configuration space, as with all Functions. PFs contain the SR-IOV Extended Capability described in § Section 9.4.3. PFs are used to discover, configure, and manage the VFs associated with the PF and for other things described in this specification.

9.4.2 SR-IOV Configuration Space §

PFs that support SR-IOV shall implement the SR-IOV Extended Capability as defined in the following sections. VFs shall implement configuration space fields and capabilities as defined in the following sections.

9.4.3 SR-IOV Extended Capability §

The SR-IOV Extended Capability defined here is a PCIe extended capability that must be implemented in each PF that supports SR-IOV. This Capability is used to describe and control a PF's SR-IOV Capabilities.

For Multi-Function Devices, each PF that supports SR-IOV shall provide the Capability structure defined in this section. This Capability structure may be present in any Function with a Type 0 Configuration Space Header. This Capability must not be present in Functions with a Type 1 Configuration Space Header.

§ Figure 9-11 shows the SR-IOV Extended Capability structure.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Byte Offset
PCI Express Extended Capability Header																																+000h
SR-IOV Capabilities Register (RO)																																+004h
SR-IOV Status Register																SR-IOV Control Register (RW)																+008h
TotalVFs (RO)																InitialVFs (RO)																+00Ch
RsvdP								Fcn Dep Link (RO)								NumVFs (RW)																+010h
VF Stride (RO)																FirstVF Offset (RO)																+014h
VF Device ID (RO)																RsvdP																+018h
Supported Page Sizes (RO)																																+01Ch
System Page Size (RW)																																+020h
VF BAR0 (RW)																																+024h
VF BAR1 (RW)																																+028h
VF BAR2 (RW)																																+02Ch
VF BAR3 (RW)																																+030h
VF BAR4 (RW)																																+034h
VF BAR5 (RW)																																+038h
VF Migration State Array Offset (RO)																																+03Ch

Figure 9-11 SR-IOV Extended Capability §

9.4.3.1 SR-IOV Extended Capability Header (Offset 00h) §

§ Table 9-2 defines the SR-IOV Extended Capability header. The Capability ID for the SR-IOV Extended Capability is 0010h.

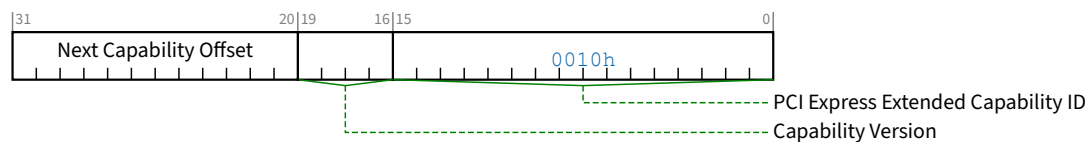


Figure 9-12 SR-IOV Extended Capability Header §

Table 9-2 SR-IOV Extended Capability Header §

Bit Location	Register Description	Attributes
15:0	PCI Express Extended Capability ID - This field is a PCI-SIG defined ID number that indicates the nature and format of the Extended Capability. The Extended Capability ID for the SR-IOV Extended Capability is 0010h.	RO
19:16	Capability Version - This field is a PCI-SIG defined version number that indicates the version of the Capability structure present. Must be 1h for this version of the specification.	RO
31:20	Next Capability Offset - This field contains the offset to the next PCI Express Capability structure or 000h if no other items exist in the linked list of Capabilities. For Extended Capabilities implemented in Configuration Space, this offset is relative to the beginning of PCI-compatible Configuration Space and thus must always be either 000h (for terminating list of Capabilities) or greater than 0FFh.	RO

9.4.3.2 SR-IOV Capabilities Register (04h) §

§ Table 9-3 defines the layout of the SR-IOV Capabilities field.

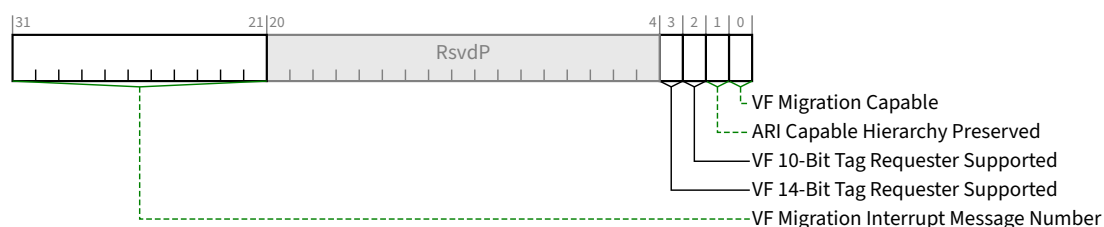


Figure 9-13 SR-IOV Capabilities Register §

Table 9-3 SR-IOV Capabilities Register §

Bit Location	Register Description	Attributes
0	VF Migration Capable - If Set, the PF is Migration Capable and operating under a Migration Capable MR-PCIM. Deprecated. New designs should hardwire this bit to 0b.	RO
1	ARI Capable Hierarchy Preserved <i>PCI Express Endpoint:</i> If Set, the ARI Capable Hierarchy bit is preserved across certain power state transitions. <i>RCiEP:</i> Not applicable - this bit <i>MUST@FLIT</i> be hardwired to 0b.	RO
2	VF 10-Bit Tag Requester Supported - If Set, all VFs associated with this PF must support 10-Bit Tag Requester capability for non-UIO Requests. If Clear, VFs must not support 10-Bit Tag Requester capability for non-UIO Requests. This bit is not applicable to a Requester when generating UIO Requests.	HwInit

Bit Location	Register Description	Attributes
	If the <u>10-Bit Tag Requester Supported</u> bit in the PF's <u>Device Capabilities 2</u> register is Clear, this bit must be Clear.	
3	VF 14-Bit Tag Requester Supported - If Set, all VFs associated with this PF must support 14-Bit Tag Requester capability for non-UIO Requests. If Clear, VFs must not support 14-Bit Tag Requester capability for non-UIO Requests. This bit is not applicable to a Requester when generating UIO Requests. If the <u>14-Bit Tag Requester Supported</u> bit in the PF's <u>Device Capabilities 3</u> register is Clear, this bit must be Clear.	<u>HwInit</u>
31:21	VF Migration Interrupt Message Number - Indicates the MSI/MSI-X vector used for migration interrupts. The value in this field is undefined if <u>VF Migration Capable</u> is Clear. VF Migration is a feature associated with the deprecated MR-IOV. New designs should hardwire this field to 0.	<u>RO</u>

9.4.3.2.1 VF Migration Capable §

VF Migration Capable is Set to indicate that the PF supports VF Migration. If Clear, the PF does not support VF Migration.

VF Migration is a feature associated with the deprecated [MR-IOV]. This bit should be hardwired to 0b in new designs.

9.4.3.2.2 ARI Capable Hierarchy Preserved §

ARI Capable Hierarchy Preserved is Set to indicate that the PF preserves the ARI Capable Hierarchy bit across certain power state transitions (see § Section 9.4.3.3.5). Components must either Set this bit or Set the No_Soft_Reset bit (see § Section 5.10.2). It is recommended that components set this bit even if they also set No_Soft_Reset.

ARI Capable Hierarchy Preserved is only present in the lowest-numbered PF of a Device (for example PF0). ARI Capable Hierarchy Preserved is Read Only Zero in other PFs of a Device.

ARI Capable Hierarchy Preserved does not apply to RCIEPs, and its value is undefined (see § Section 9.4.3.3).

9.4.3.2.3 VF Larger-Tag Requester Support §

If a PF and/or its associated VFs support UIO as a Completer, 14-bit Tags must be supported.

Otherwise, if a PF supports one or both larger-Tag Requester capabilities, its associated VFs are permitted to support the associated larger-Tag Requester capabilities as well, but this is optional. Especially for usage models where the bulk of the traffic is spread across several VFs concurrently, it may not be necessary for individual VFs to use larger Tags so they can support >256 outstanding Non-Posted Requests each.

For a given PF, it is required that either all or none of its associated VFs support larger-Tag Requester capabilities. This avoids unnecessary implementation and management complexity. See VF 14-Bit Tag Requester Supported and VF 10-Bit Tag Requester Supported.

VFs that support larger-Tag Requester capabilities have additional requirements and recommendations beyond other Function types in order to simplify error handling and reduce the possibility of larger-Tag related errors with one VF impacting other traffic.

- If one of the VF larger-Tag Requester Enable bits in the SR-IOV Control Register is Set, then each VF must use the associated larger Tags for all Non-Posted Requests that it generates. If both bits are Set at the same time, the result is undefined.
- As documented in § Table 2-11, the permitted range with 10-bit Tags enabled is 256 to 1023, and the recommended range with 14-bit Tags enabled is 1024 to 16383. The keep-out range for 10-bit Tags is Tag[9:8] being 00b, and the recommended keep-out range for 14-bit Tags is Tag[13:10] being 0000b.
- For each outstanding larger-Tag Request, if the VF receives a Completion that matches the outstanding Request other than all Tag bits in the implemented keep-out Tag bit range being zero, then the Tag is invalid, and it is strongly recommended that the VF prevent that Request from (eventually) generating a Completion Timeout error, and instead handle the error via a device-specific mechanism that avoids data corruption.

It is strongly recommended that software not configure Unexpected Completion errors to be handled as Uncorrectable Errors. This avoids them triggering System Errors or hardware error containment mechanisms like Downstream Port Containment (DPC).

IMPLEMENTATION NOTE:

NO VF LARGER-TAG COMPLETER SUPPORTED BITS §

There are no VF 14-bit or VF 10-Bit Tag Completer Supported bits. If a PF supports one or both larger-Tag Completer capabilities, then all of its associated VFs are required support the associated Tag Completer capabilities as stated in 14-Bit Tag Completer Supported and 10-Bit Tag Completer Supported. This helps avoid the complexity of PCIe hierarchies where some Completers support larger-Tag capability and some do not.

9.4.3.2.4 VF Migration Interrupt Message Number §

VF Migration is associated with the now deprecated MR-IOV. New designs should hardwire this field to 0.

9.4.3.3 SR-IOV Control Register (Offset 08h) §

§ Table 9-4 defines the layout of the SR-IOV Control fields.

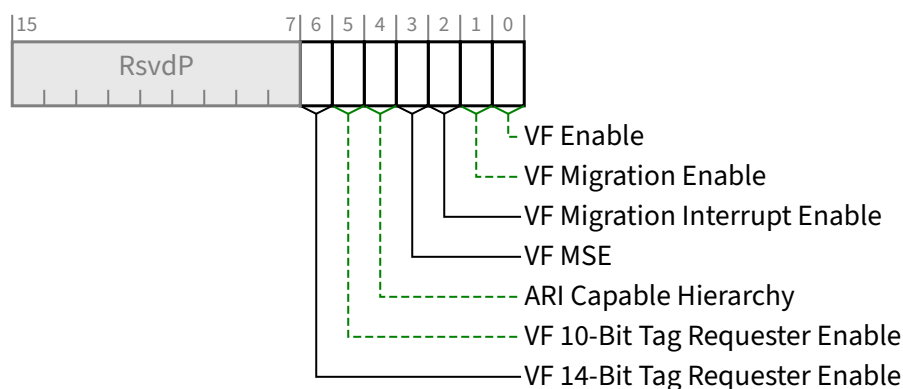


Figure 9-14 SR-IOV Control Register §

Table 9-4 SR-IOV Control Register §

Bit Location	Register Description	Attributes
0	VF Enable - Enables/Disables VFs. Default value is 0b.	RW
1	VF Migration Enable - Enables/Disables VF Migration Support. Default value is 0b. See § Section 9.4.3.3.2 .	RW or RO (see description)
2	VF Migration Interrupt Enable - Enables/Disables VF Migration State Change Interrupt. Default value is 0b.	RW
3	VF MSE - Memory Space Enable for <u>Virtual Functions</u> . Default value is 0b.	RW
4	ARI Capable Hierarchy - <i>PCI Express Endpoint:</i> This bit must be RW in the lowest-numbered PF of the Device and hardwired to 0b in all other PFs. If the value of this bit is 1b, the Device is permitted to locate VFs in Function Numbers 8 to 255 of the captured Bus Number. Otherwise, the Device must locate VFs as if it were a non-ARI Device. This bit is not affected by <u>FLR</u> of any PF or VF. Default value is 0b. <i>RCiEP:</i> Not applicable - this bit must be hardwired to 0b. Within the Root Complex, VFs are always permitted to be assigned to any Function Number allowed by <u>First VF Offset</u> and <u>VF Stride</u> rules (see § Section 9.4.3.9 and § Section 9.4.3.10).	RW or RO (see description)
5	VF 10-Bit Tag Requester Enable - If Set, all VFs must use 10-Bit Tags for all non-UIO, Non-Posted Requests they generate. If Clear, VFs must not use 10-Bit Tags for non-UIO Non-Posted Requests they generate. This bit is not applicable to a Requester when generating UIO Requests. See VF Larger-Tag Requester Support. This bit must not be Set if the <u>VF 14-Bit Tag Requester Enable</u> bit is Set; otherwise the result is undefined. If software changes the value of this bit while any VFs have outstanding Non-Posted Requests, the result is undefined.	RW or RO

Bit Location	Register Description	Attributes
	If the <u>VF 10-Bit Tag Requester Supported</u> bit in the <u>SR-IOV Capabilities register</u> is Clear, this bit is permitted to be hardwired to 0b. Default value is 0b.	
6	VF 14-Bit Tag Requester Enable - If Set, all VFs must use 14-Bit Tags for all Non-Posted Requests they generate. If Clear, VFs must not use 14-Bit Tags for Non-Posted Requests they generate. See VF Larger-Tag Requester Support. This bit must not be Set if the <u>VF 10-Bit Tag Requester Enable</u> bit is Set; otherwise the result is undefined. If software changes the value of this bit while any VFs have outstanding Non-Posted Requests, the result is undefined. If the <u>VF 14-Bit Tag Requester Supported</u> bit in the <u>SR-IOV Capabilities register</u> is Clear, this bit is permitted to be hardwired to 0b. Default value is 0b.	<u>RW</u> or <u>RO</u>

9.4.3.3.1 VF Enable §

VF Enable manages the assignment of VFs to the associated PF. If VF Enable is Set, the VFs associated with the PF are accessible in the PCI Express fabric. When Set, VFs respond to and may issue PCI Express transactions following the rules for PCI Express Endpoint Functions.

If VF Enable is Clear, VFs are disabled and not visible in the PCI Express fabric; requests to these VFs shall receive UR and these VFs shall not issue PCI Express transactions.

To allow components to perform internal initialization, after changing the VF Enable bit from Cleared to Set, the system is not permitted to issue Requests to the VFs which are enabled by that VF Enable bit until one of the following is true:

- At least 100 ms has passed
- An FRS Message has been received from the PF with a Reason Code of VF Enabled
- At least VF Enable Time has passed. VF Enable Time is either (1) the Reset Time value in the Readiness Time Reporting Extended Capability associated with the VF, or (2) a value determined by system software / firmware

207.

The Root Complex and/or system software must allow at least 1.0 s after Setting the VF Enable bit, before it may determine that a VF which fails to return a Successful Completion Status for a valid Configuration Request is broken. After Setting the VF Enable bit, the VFs enabled by that VF Enable bit are permitted to return a Configuration RRS status in response to Configuration Requests up to the 1.0 s limit, if they are not ready to provide a Successful Completion Status for a valid Configuration Request. After a PF transmits an FRS Message with a Reason Code of VF Enabled, no VF associated with that PF is permitted to return Configuration RRS in response to a Configuration Request without an intervening VF disable or other valid reset condition. After returning a Successful Completion to any Request, no VF is permitted to return Configuration RRS in response to a Configuration Request without an intervening VF disable or other valid reset condition.

Since VFs don't have an MSE bit (MSE in VFs is controlled by the VF MSE bit in the SR-IOV Extended Capability in the PF), it's possible for software to issue a Memory Request before the VF is ready to handle it. Therefore, Memory Requests must not be issued to a VF until at least one of the conditions listed in § Section 5.10.1 regarding the VF's internal initialization completing has been met.

207. For example, ACPI tables.

The VF is permitted to silently drop Memory Requests after an FLR has been issued to the VF or VF Enable has been Set in the associated PF's SR-IOV Extended Capability until the VF responds successfully to any Request (excluding the return of RRS in response to a Configuration Request).

Clearing VF Enable effectively destroys the VFs. Setting VF Enable effectively creates VFs. Setting VF Enable after it has previously been Cleared shall result in a new set of VFs. If the PF is in the D0 power state, the new VFs are in the D0_{uninitialized} state. If the PF is in a lower power state behavior is undefined (see Sections 9.6.1 and 9.6.2).

When Clearing VF Enable, a PF that supports FRS shall send an FRS Message with FRS Reason VF Disabled to indicate when this operation is complete. The PF is not permitted to send this Message if there are outstanding Non-Posted or UIO Requests issued by the PF or any of the VFs associated with the PF. The FRS Message may only be sent after these Requests have completed (or timed out).

After VF Enable is Cleared no field in the SR-IOV Extended Capability may be accessed until either:

- At least 1.0 s has elapsed after VF Enable was Cleared.
- The PF supports FRS and after VF Enable was Cleared, an FRS Message has been received from the PF with a Reason Code of VF Disabled.

If the PF has both an SR-IOV Extended Capability and an SIOV Extended Capability, setting VF Enable while SIOV Enabled is set is prohibited. In other words, SR-IOV and SIOV may not be used simultaneously in the PF.

§ Section 9.4.3.7 NumVFs, § Section 9.4.3.5 InitialVFs, § Section 9.4.3.6 TotalVFs, § Section 9.4.3.9 First VF Offset, § Section 9.4.3.13 System Page Size, and § Section 9.4.3.14 VF BARx describe additional semantics associated with this field.

9.4.3.3.2 VF Migration Enable §

VF Migration Enable must be Set to allow VF Migration on this PF.

VF Migration is associated with the now-deprecated [MR-IOV]. New designs should hardwire this bit to 0b.

9.4.3.3.3 VF Migration Interrupt Enable §

VF Migration is associated with the now-deprecated MR-IOV. New designs should hardwire this bit to 0b.

9.4.3.3.4 VF MSE (Memory Space Enable) §

VF MSE controls memory space enable for all Active VFs associated with this PF, as with the Memory Space Enable bit in a Function's PCI Command register. The default value for this bit is 0b.

When VF Enable is Set, VF memory space will respond only when VF MSE is Set. VFs shall follow the same error reporting rules as defined in the [PCIe] if an attempt is made to access a Virtual Function's memory space when VF Enable is Set and VF MSE is Clear.

IMPLEMENTATION NOTE:

VF MSE AND VF ENABLE §

VF memory space will respond with Unsupported Request when VF Enable is Clear. Thus, VF MSE is “don’t care” when VF Enable is Clear; however, software may choose to Set VF MSE after programming the VF BARn registers, prior to Setting VF Enable.

9.4.3.3.5 ARI Capable Hierarchy §

For Devices associated with an Upstream Port, ARI Capable Hierarchy is a hint to the Device that ARI has been enabled in the Root Port or Switch Downstream Port immediately above the Device. Software should set this bit to match the ARI Forwarding Enable bit in the Root Port or Switch Downstream Port immediately above the Device.

ARI Capable Hierarchy is only present in the lowest-numbered PF of a Device (for example PF₀) and affects all PFs of the Device. ARI Capable Hierarchy is Read Only Zero in other PFs of a Device.

A Device may use the setting of ARI Capable Hierarchy to determine the values for First VF Offset (see § Section 9.4.3.9) and VF Stride (see § Section 9.4.3.10). The effect of changing ARI Capable Hierarchy is undefined if VF Enable is Set in any PF. This bit must be set to its default value upon Conventional Reset. This bit is not affected by FLR of any PF or VF. If either ARI Capable Hierarchy Preserved is Set (see § Section 9.4.3.2.2) or No_Soft_Reset is Set (see § Section 5.10.2), a power state transition of this PF from D3_{Hot} to D0 does not affect the value of this bit (see § Section 5.10.2).

ARI Capable Hierarchy does not apply to RCiEPs.

IMPLEMENTATION NOTE:

ARI CAPABLE HIERARCHY §

For a Device associated with an Upstream Port, that Device has no way of knowing whether ARI has been enabled. If ARI is enabled, the Device can conserve Bus Numbers by assigning VFs to Function Numbers greater than 7 on the captured Bus Number. ARI is defined in § Section 6.13 .

Since RCiEPs are not associated with an Upstream Port, ARI does not apply, and VFs may be assigned to any Function Number within the Root Complex permitted by First VF Offset and VF Stride (see § Section 9.4.3.8 and § Section 9.4.3.9).

9.4.3.4 SR-IOV Status Register (Offset 0Ah) §

§ Table 9-5: defines the layout of the SR-IOV Status field.

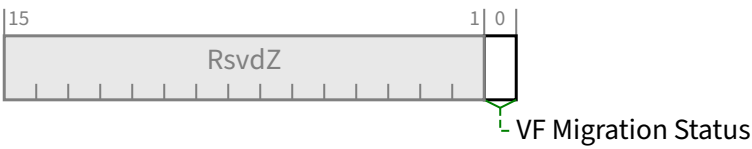


Figure 9-15 SR-IOV Status §

Table 9-5 SR-IOV Status §

Bit Location	Register Description	Attributes
0	VF Migration Status - VF Migration is associated with the now-deprecated [MR-IOV]. New designs should hardwire this bit to 0b.	RW1C

9.4.3.4.1 VF Migration Status §

VF Migration is associated with the now-deprecated [MR-IOV]. New designs should hardwire this bit to 0b.

9.4.3.5 InitialVFs (Offset 0Ch) §

VF Migration is associated with the now deprecated MR-IOV. New designs should encode this field as HwInit with a value equal to TotalVFs.

9.4.3.6 TotalVFs (Offset 0Eh) §

TotalVFs indicates the maximum number of VFs that are associated with the PF.

The minimum value of TotalVFs is 0.

This field is HwInit and must contain the same value as InitialVFs.

9.4.3.7 NumVFs (Offset 10h) §

NumVFs controls the number of VFs that are visible. SR-PCIM sets NumVFs as part of the process of creating VFs. This number of VFs shall be visible in the PCI Express fabric after both NumVFs is set to a valid value and VF Enable is Set.

The results are undefined if NumVFs is set to a value greater than TotalVFs.

NumVFs may only be written while VF Enable is Clear. If NumVFs is written when VF Enable is Set, the results are undefined.

The initial value of NumVFs is undefined.

9.4.3.8 Function Dependency Link (Offset 12h) §

The programming model for a Device may have vendor-specific dependencies between sets of Functions. The Function Dependency Link field is used to describe these dependencies.

This field describes dependencies between PFs. VF dependencies are the same as the dependencies of their associated PFs.

If a PF is independent from other PFs of a Device, this field shall contain its own Function Number.

If a PF is dependent on other PFs of a Device, this field shall contain the Function Number of the next PF in the same Function Dependency List. The last PF in a Function Dependency List shall contain the Function Number of the first PF in the Function Dependency List.

If PF_p and PF_q are in the same Function Dependency List, then any SI that is assigned $VF_{p,n}$ shall also be assigned to $VF_{q,n}$.

IMPLEMENTATION NOTE:

FUNCTION DEPENDENCY LINK EXAMPLE §

Consider the following scenario:

SR-IOV Field	PF 0	PF 1	PF 2
Function Dependency Link	1	0	2
NumVFs	4	4	6
First VF Offset	4	4	4
VF Stride	3	3	3

Function Number	Description	Independent
0	PF 0	No
1	PF 1	No
2	PF 2	Yes
3	Function not present	
4	VF 0,1 (aka PF 0 VF 1)	No
5	VF 1,1 (aka PF 1 VF 1)	No
6	VF 2,1 (aka PF 2 VF 1)	Yes
7	VF 0,2	No
8	VF 1,2	No
9	VF 2,2	Yes
10	VF 0,3	No
11	VF 1,3	No
12	VF 2,3	Yes
13	VF 0,4	No
14	VF 1,4	No
15	VF 2,4	Yes
16 to 17	Functions not present	
18	VF 2,5	Yes
19 to 20	Functions not present	
21	VF 2,6	Yes

Function Number	Description	Independent
22 to 255	Functions not present	

In this example, Functions 4 and 5 must be assigned to the same SI. Similarly, Functions 7 and 8, 10 and 11, and 13 and 14 must be assigned together. If PFs are assigned to SIs, Functions 0 and 1 must be assigned together as well. Functions 2, 6, 9, 12, 15, 18, and 21 are independent and may be assigned to SIs in any fashion.

All PFs in a Function Dependency List shall have the same values for the InitialVFs and TotalVFs fields.

SR-PCIM shall ensure that all PFs in a Function Dependency List have the same values for the NumVFs and VF Enable fields before any VF in that Function Dependency List is assigned to an SI.

VF Mapping operations occur independently for every VF. SR-PCIM shall not assign a VF to an SI until it can assign all dependent VFs.

Similarly, SR-PCIM shall not remove a VF from an SI until it can remove all dependent VFs.

9.4.3.9 First VF Offset (Offset 14h) §

First VF Offset is a constant and defines the Routing ID offset of the first VF that is associated with the PF that contains this Capability structure. The first VF's 16-bit Routing ID is calculated by adding the contents of this field to the Routing ID of the PF containing this field ignoring any carry, using unsigned, 16-bit arithmetic.

A VF shall not be located on a Bus Number that is numerically smaller than its associated PF.

This field may change value when the lowest-numbered PF's ARI Capable Hierarchy value changes or when this PF's NumVFs value changes.

Note: First VF Offset is unused if NumVFs is 0. If NumVFs is greater than 0, First VF Offset must not be zero.

9.4.3.10 VF Stride (Offset 16h) §

VF Stride defines the Routing ID offset from one VF to the next one for all VFs associated with the PF that contains this Capability structure. The next VF's 16-bit Routing ID is calculated by adding the contents of this field to the Routing ID of the current VF, ignoring any carry, using unsigned 16-bit arithmetic.

This field may change value when the lowest-numbered PF's ARI Capable Hierarchy value changes or when this PF's NumVFs value changes.

Note: VF Stride is unused if NumVFs is 0 or 1. If NumVFs is greater than 1, VF Stride must not be zero.

9.4.3.11 VF Device ID (Offset 1Ah) §

This field contains the Device ID that should be presented for every VF to the SI.

VF Device ID may be different from the PF Device ID. A VF Device ID must be managed by the vendor. The vendor must ensure that the chosen VF Device ID does not result in the use of an incompatible device driver.

9.4.3.12 Supported Page Sizes (Offset 1Ch) §

This field indicates the page sizes supported by the PF. This PF supports a page size of 2^{n+12} if bit n is Set. For example, if bit 0 is Set, the PF supports 4-KB page sizes. PFs are required to support 4-KB, 8-KB, 64-KB, 256-KB, 1-MB, and 4-MB page sizes. All other page sizes are optional.

The page size describes the minimum alignment requirements for VF BAR resources as described in § Section 9.4.3.13 .

9.4.3.13 System Page Size (Offset 20h) §

This field defines the page size the system will use to map the VFs' memory addresses. Software must set the value of the System Page Size to one of the page sizes set in the Supported Page Sizes field (see § Section 9.4.3.12). As with Supported Page Sizes, if bit n is Set in System Page Size, the VFs associated with this PF are required to support a page size of 2^{n+12} . For example, if bit 1 is Set, the system is using an 8-KB page size. The results are undefined if System Page Size is zero. The results are undefined if more than one bit is Set in System Page Size. The results are undefined if a bit is Set in System Page Size that is not Set in Supported Page Sizes.

When System Page Size is set, the VF associated with this PF is required to align all BAR resources on a System Page Size boundary. Each VF BAR n or VF BAR n pair (see § Section 9.4.3.14) shall be aligned on a System Page Size boundary. Each VF BAR n or VF BAR n pair defining a non-zero address space shall be sized to consume an integer multiple of System Page Size bytes. All data structures requiring page size alignment within a VF shall be aligned on a System Page Size boundary.

VF Enable must be zero when System Page Size is written. The results are undefined if System Page Size is written when VF Enable is Set.

Default value is 0000 0001h (i.e., 4 KB).

9.4.3.14 VF BAR0 (Offset 24h), VF BAR1 (Offset 28h), VF BAR2 (Offset 2Ch), VF BAR3 (Offset 30h), VF BAR4 (Offset 34h), VF BAR5 (Offset 38h) §

These fields must define the VF's Base Address Registers (BARs). These fields behave as normal PCI BARs, as described in § Section 7.5.1 . They can be sized by writing all 1s and reading back the contents of the BARs as described in § Section 7.5.1.2.1 , complying with the low order bits that define the BAR type fields.

These fields may have their attributes affected by the VF Resizable BAR Extended Capability (see § Section 7.8.7) if it is implemented.

The amount of address space decoded by each BAR shall be an integral multiple of System Page Size.

Each VF BAR n , when "sized" by writing 1s and reading back the contents, describes the amount of address space consumed and alignment required by a single Virtual Function, per BAR. When written with an actual address value, and VF Enable and VF MSE are Set, the BAR maps NumVFs BARs. In other words, the base address is the address of the first VF BAR n associated with this PF and all subsequent VF BAR n address ranges follow as described below.

VF BARs shall only support 32-bit and 64-bit memory space. PCI I/O Space is not supported in VFs. Bit 0 of any implemented VF BAR x must be RO 0b except for a VF BAR x used to map the upper 32 bits of a 64-bit memory VF BAR pair.

The alignment requirement and size read is for a single VF, but when VF Enable is Set and VF MSE is Set, the BAR contains the base address for all (NumVFs) VF BAR n .

The algorithm to determine the amount of address space mapped by a VF BAR_{*n*} differs from the standard BAR algorithm as follows:

1. Resize the BAR via the VF Resizable BAR Extended Capability (see § Section 7.8.7) if it is implemented.
2. After reading the low order bits to determine if the BAR is a 32-bit BAR or 64-bit BAR pair, determine the size and alignment requirements by writing all 1s to VF BAR_{*n*} (or VF BAR_{*n*} and VF BAR_{*n*+1} for a 64-bit BAR pair) and reading back the contents of the BAR or BAR pair. Convert the bit mask returned by the read(s) to a size and alignment value as described in § Section 7.5.1.2.1 . This value is the size and alignment for a single VF.
3. Multiply the value from step 2 by the value set in NumVFs to determine the total amount of space the BAR or BAR pair will map after VF Enable and VF MSE are Set.

For each VF BAR_{*n*} field, *n* corresponds to one of the VFs BAR spaces. § Table 9-8 shows the relationship between *n* and a Function's BAR.

Table 9-8 BAR Offsets §

<i>n</i>	BAR Offset in a Type 0 Header
0	10h
1	14h
2	18h
3	1Ch
4	20h
5	24h

The contents of all VF BAR_{*n*} registers are indeterminate after System Page Size is changed.

9.4.3.15 VF Migration State Array Offset (Deprecated) (Offset 3Ch) §

VF Migration is associated with the now deprecated [MR-IOV]. New designs should hardwire this register to 0000 0000h.

9.4.4 PF/VF Configuration Space Header §

This section's material in previous versions of this specification has been integrated into § Section 7.5.1 .

9.4.5 PCI Express Capability Changes §

This section's material in previous versions of this specification has been integrated into § Section 7.5.3 .

9.4.6 PCI Standard Capabilities §

SR-IOV usage of PCI Standard Capabilities is described in § Table 9-9. Items marked n/a are not applicable to PFs or VFs.

Table 9-9 SR-IOV Usage of PCI Standard Capabilities §

Capability ID	Description	PF Attributes	VF Attributes
00h	Null Capability	Base	Base
01h	PCI Power Management Interface	Base	Optional. See § Section 5.10 .
02h	AGP	n/a	n/a
03h	VPD	Base	Optional. See § Section 7.9.18 .
04h	Slot Identification	n/a	n/a
05h	MSI	Base	See § Section 9.2.1.4 .
06h	CompactPCI Hot Swap	n/a	n/a
07h	PCI-X	n/a	n/a
08h	HyperTransport	n/a	n/a
09h	Vendor-specific	Base	Base
0Ah	Debug Port	Base	Base
0Bh	CompactPCI Central Resource Control	n/a	n/a
0Ch	PCI Hot Plug	Base	n/a
0Dh	PCI Bridge Subsystem ID	n/a	n/a
0Eh	AGP 8x	n/a	n/a
0Fh	Secure Device	n/a	n/a
10h	PCI Express	Base	See § Section 9.4.5 .
11h	MSI-X	See § Section 9.2.1.4 .	See § Section 9.2.1.4 .
12h	Serial ATA Data/Index Configuration	Base	n/a
13h	Advanced Features	n/a	n/a
14h	Enhanced Allocation	Base	Must not implement.
15h	Flattening Portal Bridge (FPB)	n/a	n/a

9.4.7 PCI Express Extended Capabilities Changes §

SR-IOV usage of PCI Express Extended Capabilities is described in § Table 9-10. Items marked n/a are not applicable to PFs or VFs (e.g., for capabilities only present in Root Complexes, only present in Function 0, or only for managing a Port).

Table 9-10 SR-IOV Usage of PCI Express Extended Capabilities §

Extended Capability ID	Description	PF Attributes	VF Attributes
0000h	Null Capability	Base	Base
0001h	Advanced Error Reporting Extended Capability (AER)	Base	See capability description

Extended Capability ID	Description	PF Attributes	VF Attributes
0002h	<u>Virtual Channel Extended Capability (02h)</u>	Base	Must not implement. See capability description
0003h	<u>Device Serial Number Extended Capability</u>	Base	See capability description
0004h	<u>Power Budgeting Extended Capability</u>	Base	Must not implement. See capability description
0005h	<u>Root Complex Link Declaration Extended Capability</u>	n/a	n/a
0006h	<u>Root Complex Internal Link Control Extended Capability</u>	n/a	n/a
0007h	<u>Root Complex Event Collector Endpoint Association Extended Capability</u>	n/a	n/a
0008h	<u>Multi-Function Virtual Channel Extended Capability</u>	Base	n/a; only present in Function 0
0009h	<u>Virtual Channel Extended Capability (09h)</u>	Base	Must not implement. See capability description
000Ah	<u>RCRB Header Extended Capability</u>	n/a	n/a
000Bh	<u>Vendor-specific Extended Capability</u>	Base	Base
000Ch	Deprecated; formerly used for the Configuration Access Correlation Extended Capability	n/a	n/a
000Dh	<u>ACS Extended Capability</u>	See capability description	See capability description
000Eh	<u>ARI Extended Capability (ARI)</u>	See capability description	See capability description
000Fh	<u>ATS Extended Capability</u>	See capability description	See capability description
0010h	<u>SR-IOV Extended Capability</u>	See capability description	Must not implement. See capability description
0011h	Deprecated; formerly used for the MR-IOV Extended Capability (MR-IOV)	n/a	n/a
0012h	<u>Multicast Extended Capability</u>	See capability description	See capability description
0013h	<u>Page Request Extended Capability (PRI)</u>	See capability description	See capability description
0014h	Reserved for AMD	Base	Base
0015h	<u>Resizable BAR Extended Capability</u>	Base	Must not implement. See capability description
0016h	<u>Dynamic Power Allocation Extended Capability (DPA)</u>	See capability description	Must not implement. See capability description
0017h	<u>TPH Requester Extended Capability (TPH)</u>	See capability description	See capability description

Extended Capability ID	Description	PF Attributes	VF Attributes
0018h	<u>LTR Extended Capability</u>	Base	n/a; only present in Function 0
0019h	<u>Secondary PCI Express Extended Capability</u>	Base	n/a; only present in Function 0
001Ah	<u>PMUX Extended Capability</u>	Base	n/a; only for managing a Port
001Bh	<u>PASID Extended Capability</u>	Base	Must not implement
001Ch	Deprecated; formerly used for the LN Requester Extended Capability	n/a	n/a
001Dh	<u>DPC Extended Capability</u>	n/a; only for managing a Port	n/a; only for managing a Port
001Eh	<u>L1 PM Substates Extended Capability</u>	Base	n/a; only present in Function 0
001Fh	<u>Precision Time Measurement Extended Capability (PTM)</u>	Base	n/a; only for managing a Port
0020h	<u>PCI Express over M-PHY Extended Capability (M-PCIe)</u>	Base	n/a; only for managing a Port
0021h	<u>FRS Queueing Extended Capability</u>	n/a	n/a
0022h	<u>Readiness Time Reporting Extended Capability</u>	Base	See capability description
0023h	<u>Designated vendor-specific Extended Capability</u>	Base	Base
0024h	<u>VF Resizable BAR Extended Capability</u>	See capability description	Must not implement. See capability description
0025h	<u>Data Link Feature Extended Capability</u>	Base	n/a; only for managing a Port
0026h	<u>Physical Layer 16.0 GT/s Extended Capability</u>	Base	n/a; only for managing a Port
0027h	<u>Lane Margining at the Receiver Extended Capability</u>	Base	n/a; only for managing a Port
0028h	<u>Hierarchy ID Extended Capability</u>	Base	Base
0029h	<u>Native PCIe Enclosure Management Extended Capability (NPEM)</u>	Base	n/a; only for managing a Port
002Ah	<u>Physical Layer 32.0 GT/s Extended Capability</u>	Base	Must not implement
002Bh	<u>Alternate Protocol Extended Capability</u>	Base	Must not implement
002Ch	<u>SFI Extended Capability (System Firmware Intermediary)</u>	Base	Must not implement
002Dh	<u>Shadow Functions Extended Capability</u>	Base	Base
002Eh	<u>Data Object Exchange Extended Capability</u>	Base	Base
002Fh	<u>Device 3 Extended Capability</u>	Base	Base
0030h	<u>IDE Extended Capability</u>	Base	n/a; only present in Function 0
0031h	<u>Physical Layer 64.0 GT/s Extended Capability</u>	Base	Must not implement
0032h	<u>Flit Logging Extended Capability</u>	Base	Must not implement
0033h	<u>Flit Performance Measurement Extended Capability</u>	Base	Must not implement

Extended Capability ID	Description	PF Attributes	VF Attributes
0034h	<u>Flit Error Injection Extended Capability</u>	Base	Must not implement
0035h	<u>Streamlined Virtual Channel Extended Capability (SVC)</u>	Base	Must not implement. See capability description
0036h	<u>MMIO Register Block Locator Extended Capability (MRBL)</u>	Base	Base
0037h	<u>NOP Flit Extended Capability</u>	Base	Must not implement
0038h	<u>SIOV Extended Capability</u>	See capability description	Must not implement See capability description

9.5 SIOV Configuration §

9.5.1 SIOV Configuration Overview §

This section provides SIOV-added requirements for implementing PFs.

PFs are discoverable in configuration space, as with all Functions. PFs contain the SIOV Extended Capability.

9.5.2 SIOV Configuration Space §

PFs that support SIOV shall implement the SIOV Extended Capability as defined in the following sections. Scalable Device Interfaces (SDIs) implement no Configuration Space.

9.5.3 SIOV Extended Capability §

The SIOV Extended Capability defined here is a PCIe Extended Capability that must be implemented in each PF that supports SIOV. This Capability is used to describe and control a PF's SIOV Capabilities.

For Multi-Function Devices, each PF that supports SIOV shall provide the capability structure defined in this section. This capability structure may be present in any Function with a Type 0 Configuration Space Header. This Capability must not be present in Functions with a Type 1 Configuration Space Header.

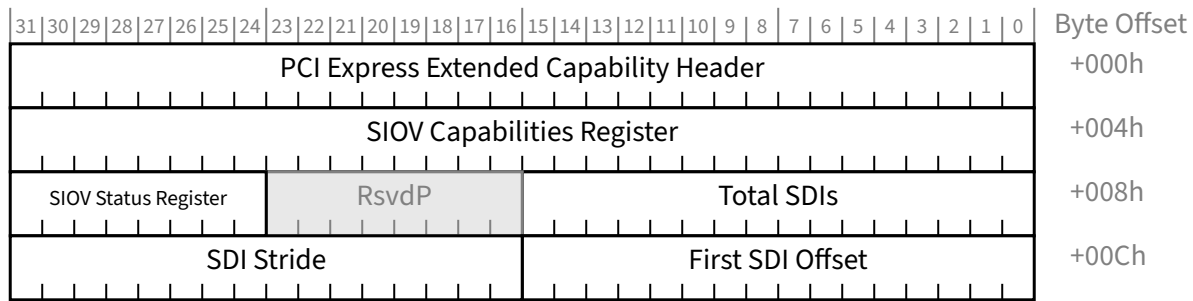


Figure 9-16 SIOV Extended Capability §

9.5.4 SIOV Extended Capability Header (Offset 00h) §

§ Table 9-11 defines the SIOV Extended Capability header. The Capability ID for the SIOV Extended Capability is 0038h.

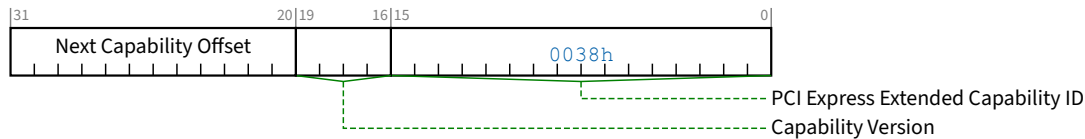


Figure 9-17 SIOV Extended Capability Header §

Table 9-11 SIOV Extended Capability Header §

Bit Location	Register Description	Attributes
15:0	PCI Express Extended Capability ID - This field is a PCI-SIG defined ID number that indicates the nature and format of the Extended Capability. The Extended Capability ID for the SIOV Extended Capability is 0038h.	RO
19:16	Capability Version - This field is a PCI-SIG defined version number that indicates the version of the capability structure present. Must be 1h for this version of the specification.	RO
31:20	Next Capability Offset - This field contains the offset to the next PCI Express Capability structure or 000h if no other items exist in the linked list of Capabilities. For Extended Capabilities implemented in Configuration Space, this offset is relative to the beginning of PCI-compatible Configuration Space and thus must always be either 000h (for terminating list of Capabilities) or greater than 0FFh.	RO

9.5.5 SIOV Capabilities Register (04h) §

§ Table 9-12 defines the layout of the SIOV Capabilities register.

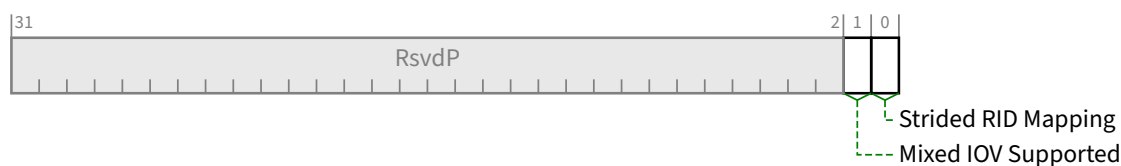


Figure 9-18 SIOV Capabilities Register §

Table 9-12 SIOV Capabilities Register §

Bit Location	Register Description	Attributes
0	Strided RID Mapping – This PF uses the Strided RID Mapping. The only RID that may be assigned to an SDI is the one that is implied by First SDI Offset and SDI Stride. See § Section 9.1.1 .	RO
1	Mixed IOV Supported – If no other PF exists in this device, or if no other PF supports <u>SR-IOV</u> , then this bit has no meaning and is hardwired to 0. If another PF in this device exists, and this bit is 0, then software that enables IOV must choose either SR-IOV (setting VF Enable) for all PFs within the device or choose SIOV (enabling SDIs) for all PFs. If this bit is 1, software may choose to enable <u>SR-IOV</u> on some PFs and <u>SIOV</u> on others.	RO

9.5.6 Total SDIs (Offset 08h) §

This field is RO and indicates the total number of SDIs supported by this Physical Function. Software can use this field to determine the total number of RIDs that this Physical Function would need for its SDIs, if all were simultaneously in use.

9.5.7 SIOV Status Register (Offset 0Bh) §

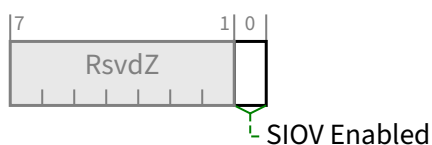


Figure 9-19 SIOV Status Register §

Table 9-13 SIOV Status Register §

Bit Location	Register Description	Attributes
0	SIOV Enabled – This bit indicates that at least one SDI is enabled (through device-specific means). This bit is Cleared when all SDIs are disabled. System software may use this bit to determine that a RID associated with an SDI is in active use. If system software finds this bit Set during system boot, or after system reset, a Function-Level Reset may be required to ensure that no SDI is active. Default value of this bit is 0.	RO

9.5.8 First SDI Offset (Offset 0Ch) §

This field is RO and valid only when the Strided RID Mapping bit is Set in the SIOV Capabilities Register. If that bit is Clear, this field is RsvdP.

First SDI Offset is a constant and defines the RID offset of the first SDI that is associated with the PF that contains this capability structure. The first SDI's RID is calculated by adding the contents of this field to the RID of the PF containing this field ignoring any carry, using unsigned, 16-bit arithmetic.

9.5.9 SDI Stride (Offset 0Eh) §

This field is RO and valid only when the Strided RID Mapping bit is Set in the SIOV Capabilities Register. If that bit is Clear, this field is RsvdP.

SDI Stride defines the RID offset from one SDI to the next one for all SDIs associated with the PF that contains this capability structure. The next SDI's RID is calculated by adding the contents of this field to the RID of the current SDI, ignoring any carry, using unsigned 16-bit arithmetic.

If Strided RID Mapping is Set, SDI Stride must not be Zero.

